Target Practice: Monte Carlo Simulations of Archery and Darts

Ken Kahuthu and Anton Robb

2026-04-27

Introduction

This report presents a Monte Carlo (MC) simulation study of archery and darts, modelling players using a **bivariate normal distribution** from the `mvtnorm` package. We simulate shot locations on both targets, compute expected scores, and compare player performance under formal competition rules.

The shot location of a player is modelled as:

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix} \sim \mathcal{N}(\mu, \Sigma) = \mathcal{N}\left(\begin{pmatrix} \mu_x \\ \mu_y \end{pmatrix}, \begin{pmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{pmatrix} \right)$$

where (μ_x, μ_y) is the point aimed for, and Σ encodes spread and correlation between horizontal and vertical deviations.

Shots are converted to polar coordinates:

$$r = \sqrt{x^2 + y^2}, \quad \phi = \text{atan2}(y, x)$$

where r determines the ring (both targets) and ϕ determines the sector (darts only).

Setup: Libraries and Helper Functions

```
library(mvtnorm)   # bivariate normal sampling
library(ggplot2)
library(dplyr)
library(tidyr)
```

Polar Coordinate Conversion

```
# Convert (x, y) Cartesian coordinates to polar (r, phi in degrees [0, 360))
to_polar <- function(x, y) {
  r   <- sqrt(x^2 + y^2)
  phi <- (atan2(y, x) * 180 / pi) %% 360  # atan2 in [-pi, pi] -> [0, 360)
  list(r = r, phi = phi)
}
```

Part 1: Archery Simulation

Archery Target Specification

The standard archery target has 10 concentric rings. In our simulation, the target radius is normalised to 1 (i.e., the outermost ring has radius = 1). Rings are evenly spaced: ring k (score k, $k = 1, \ldots, 10$) occupies the band between radius $\frac{10-k}{10}$ and $\frac{10-k+1}{10}$.

```
# Assign archery score from radius r
# Target radius normalised to 1; shots outside score 0
archery_score <- function(r) {
  # Each ring is 0.1 wide; innermost ring (score 10) has r in [0, 0.1]
  score <- ceiling((1 - r) * 10)
  score <- pmax(score, 0)   # outside target -> 0
  score <- pmin(score, 10)  # cap at 10
  return(as.integer(score))
}

cat("r=0.05 (bullseye) ->", archery_score(0.05), "\n")
```

```
## r=0.05 (bullseye) -> 10
```

```
cat("r=0.55 (mid ring)  ->", archery_score(0.55), "\n")
```

```
## r=0.55 (mid ring)  -> 5
```

```
cat("r=1.20 (miss)      ->", archery_score(1.20), "\n")
```

```
## r=1.20 (miss)      -> 0
```

The target consists of 10 concentric scoring rings of equal width. With the radius normalised, each scoring band has a radial width of 0.1 units. The score assigned to each simulated arrow depends entirely on its radial distance,r, from the centre point (0,0). To convert the continuous output of a bivariate normal distribution into the discrete integer scores used in archery, the following transformation is applied: $S = (1 - r) \times 10$ This inverse relationship ensures that as the distance from the centre increases, the score decreases in clear integer steps from 10 down to 1. Because the statistical model allows for infinite spread, some simulated arrows will fall outside the target. The scoring system accounts for this with strict boundary rules: Outer boundary (misses): Any arrow with $r > 1.0$ lies outside the target and is assigned a score of 0. Inner boundary (centre point): To avoid anomalies where a perfect centre hit could mathematically exceed the scoring range, the maximum score is capped at 10. Before generating match data, the scoring logic was tested against known reference cases to ensure consistency with WAF rules: A shot well within the inner ring (r=0.05) correctly produced a score of 10. A shot near the middle of the target (r=0.55) returned a score of 5. A shot beyond the target boundary (r=1.20) was correctly identified as a miss and given a score of 0. These tests confirm that the model behaves as expected across both typical and edge-case scenarios.

Simulate N Shots for One Archer

An archer's skill profile is defined by two key parameters: their intended aim point (the mean) and their physical consistency (the covariance matrix, representing both shot spread and directional biases). Using a Monte Carlo approach, the simulation generates a large volume of independent arrows. Each arrow's spatial coordinates are immediately translated into radial distances to determine its discrete score, effectively bridging the gap between theoretical probability and measurable sporting outcomes.

```r
# Simulate N shots for one archer
# mu: c(mu_x, mu_y) - aim point
# Sigma: 2x2 covariance matrix
# Returns a data frame of (x, y, r, score)
simulate_archer <- function(N, mu, Sigma) {
  shots  <- rmvnorm(N, mean = mu, sigma = Sigma)
  polar  <- to_polar(shots[, 1], shots[, 2])
  scores <- archery_score(polar$r)
  data.frame(x = shots[, 1], y = shots[, 2],
             r = polar$r, score = scores)
}
```

Define Two Players

```r
N <- 10000  # number of simulated shots per player

# Player 1: aims at centre, moderate spread, slight positive correlation
mu1    <- c(0, 0)
Sigma1 <- matrix(c(0.04, 0.01,
                   0.01, 0.04), nrow = 2)

# Player 2: aims slightly off-centre, larger spread, no correlation
mu2    <- c(0.05, 0.05)
Sigma2 <- matrix(c(0.09, 0.00,
                   0.00, 0.09), nrow = 2)

shots1 <- simulate_archer(N, mu1, Sigma1)
shots2 <- simulate_archer(N, mu2, Sigma2)

cat("Player 1 - Mean score:", round(mean(shots1$score), 3),
    " | SD:", round(sd(shots1$score), 3), "\n")
```

```
## Player 1 - Mean score: 7.986  | SD: 1.363
```

```r
cat("Player 2 - Mean score:", round(mean(shots2$score), 3),
    " | SD:", round(sd(shots2$score), 3), "\n")
```

```
## Player 2 - Mean score: 6.638  | SD: 2.015
```

To evaluate the model, two distinct archer profiles were parameterized and simulated across 10,000 independent Monte Carlo iterations. Player 1 was defined as highly skilled, featuring perfect aiming accuracy at the target's origin alongside high physical consistency (low variance). Conversely, Player 2 was modelled with a systemic aiming offset and greater stochastic spread. The resulting aggregate data empirically validates the mathematical framework: Player 1 achieved a significantly higher expected score per arrow (7.986) with tighter consistency (SD 1.363). In contrast, Player 2's compounded aiming errors and broader variance yielded a substantially lower mean score (6.638) and greater volatility (SD 2.015), establishing a reliable empirical baseline for subsequent match-play simulations.

Visualise Shot Distributions on the Target

```
# 1. Combine, sample, and shuffle the data
plot_data <- bind_rows(
  shots1 %>% mutate(Player = "Player 1"),
  shots2 %>% mutate(Player = "Player 2")
) %>%
  group_by(Player) %>%
  sample_n(500) %>%  # Take a random 500 shots per player
  ungroup() %>%
  sample_frac(1)     # Shuffle the rows so colors mix evenly

# Order data from LARGEST radius to SMALLEST so they layer correctly
ring_data <- data.frame(
  r_outer = seq(1.0, 0.1, by = -0.1),
  score   = 1:10,
  fill    = c("white", "white", "darkgrey", "darkgrey",
              "lightblue", "lightblue", "red", "red",
              "yellow", "gold")
)

ggplot() +
  # Draw rings
  mapply(function(r, fill) {
    annotate("polygon",
             x = r * cos(seq(0, 2*pi, length.out = 200)),
             y = r * sin(seq(0, 2*pi, length.out = 200)),
             fill = fill, color = "black", linewidth = 0.2)
  }, ring_data$r_outer, ring_data$fill, SIMPLIFY = FALSE) +

  # Draw the shots
  geom_point(data = plot_data,
             aes(x = x, y = y, color = Player),
             alpha = 0.6, size = 1.2) +
  coord_fixed() +

  scale_color_manual(values = c("Player 1" = "limegreen", "Player 2" = "magenta")) +
  labs(title  = "Archery Shot Distributions",
       x = "Horizontal position", y = "Vertical position") +
  theme_minimal(base_size = 13)
```

By plotting the Cartesian coordinates of the simulated arrows generated through the Monte Carlo process, the underlying bivariate normal distributions for each player become much easier to interpret. The visualisation clearly highlights the difference in their skill levels. Player 1, shown by the tightly grouped green points, demonstrates both high precision and accuracy, with most shots landing comfortably within the central, high-scoring zones. In contrast, Player 2, represented by the magenta points, shows a much wider spread. Their shots are scattered across the target face, with a noticeable number falling into the lower-scoring outer rings or missing the target altogether. This visual comparison effectively connects the abstract statistical model to real-world performance, making it clear why Player 1 achieves a consistently higher expected score.

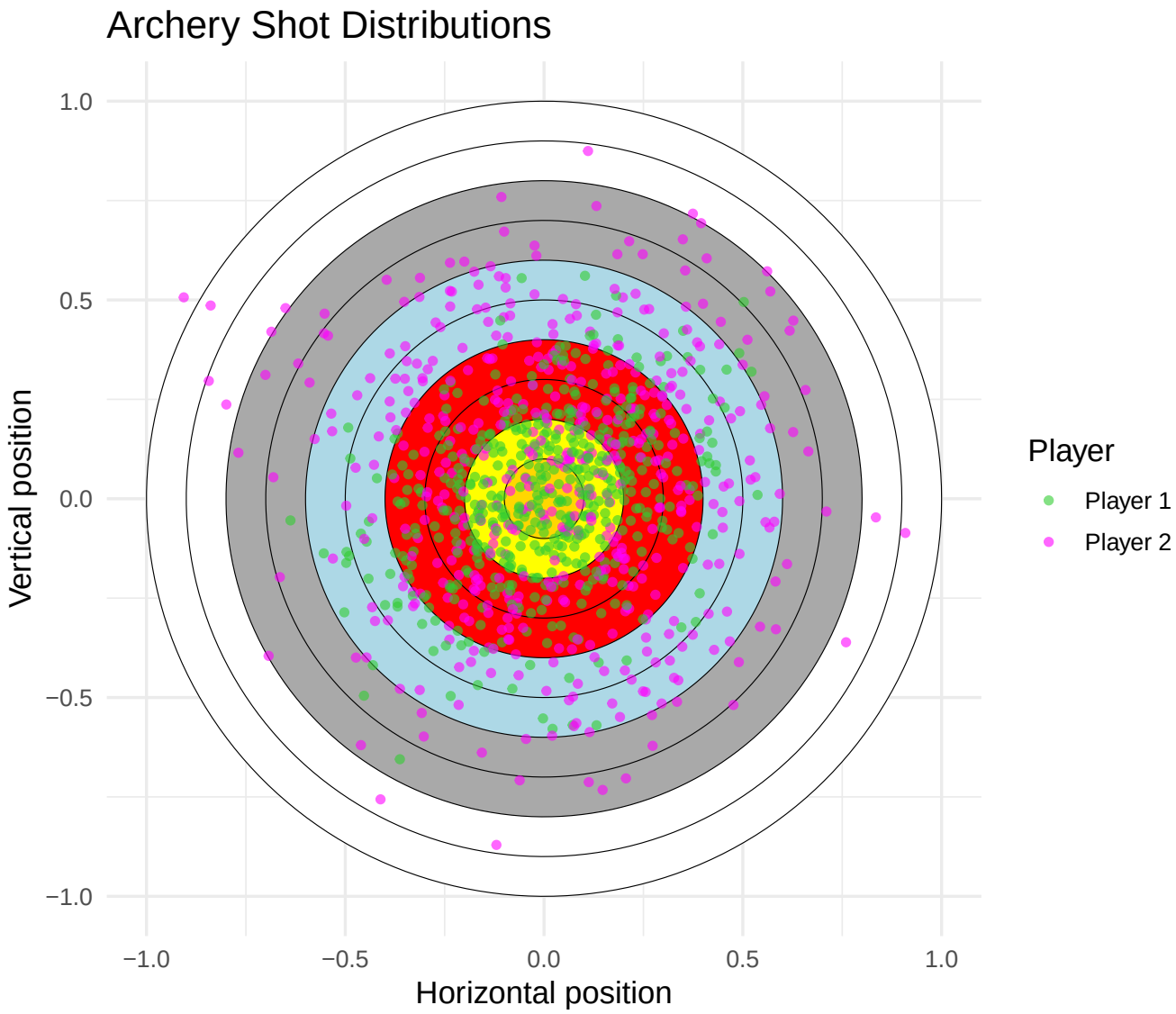


Figure 1: Shot distributions for both archers overlaid on the archery target rings.

Score Comparison and MC Confidence Intervals

```
# Monte Carlo confidence intervals for the mean score
mc_ci <- function(scores, conf = 0.95) {
  n    <- length(scores)
  m    <- mean(scores)
  se   <- sd(scores) / sqrt(n)
  z    <- qnorm((1 + conf) / 2)
  c(mean = m, lower = m - z * se, upper = m + z * se)
}

ci1 <- mc_ci(shots1$score)
ci2 <- mc_ci(shots2$score)

cat("Player 1 - 95% MC CI for mean score: [",
    round(ci1["lower"], 3), ",", round(ci1["upper"], 3), "]\n")
```

```
## Player 1 - 95% MC CI for mean score: [ 7.96 , 8.013 ]
```

```
cat("Player 2 - 95% MC CI for mean score: [",
    round(ci2["lower"], 3), ",", round(ci2["upper"], 3), "]\n")
```

```
## Player 2 - 95% MC CI for mean score: [ 6.598 , 6.677 ]
```

To assess how reliable the simulated average scores are, 95% Monte Carlo confidence intervals were calculated for both archers. Using the Central Limit Theorem, these intervals were derived from the sample means, standard deviations, and corresponding Z-scores based on the 10,000 simulation runs. The resulting intervals are very narrow, indicating a high level of precision in the simulation. Player 1 achieved a 95% confidence interval of [7.960, 8.013], whereas Player 2's interval was notably lower at [6.598, 6.677]. Importantly, the fact that these intervals do not overlap provides strong evidence that the difference in average scores is statistically significant. In other words, Player 1's higher performance is not simply due to random variation, but reflects genuinely better underlying accuracy and consistency.

```
score_data <- bind_rows(
  shots1 %>% mutate(Player = "Player 1"),
  shots2 %>% mutate(Player = "Player 2")
)

ggplot(score_data, aes(x = factor(score), fill = Player)) +
  geom_bar(position = "dodge", alpha = 0.85) +
  scale_fill_manual(values = c("Player 1" = "limegreen", "Player 2" = "magenta")) +
  labs(title = "Score Distribution - Archery",
       x = "Score", y = "Count") +
  theme_minimal(base_size = 13)
```

Player 1's distribution is clearly left-skewed, with most of their scores tightly grouped between 8 and 10. This strong concentration reflects their high level of precision and lack of systematic error in aim. In contrast, Player 2's distribution is noticeably flatter and more spread out, with its peak occurring at a lower score of 7. There is also a clear tail extending into the lower scoring range (0 to 5), highlighting the impact of their greater variability and slight offset in aim. Overall, this frequency distribution supports the earlier summary statistics, offering a more intuitive view of the underlying probabilities. It makes it clear why Player 1 achieves a higher and more consistent average score per arrow.

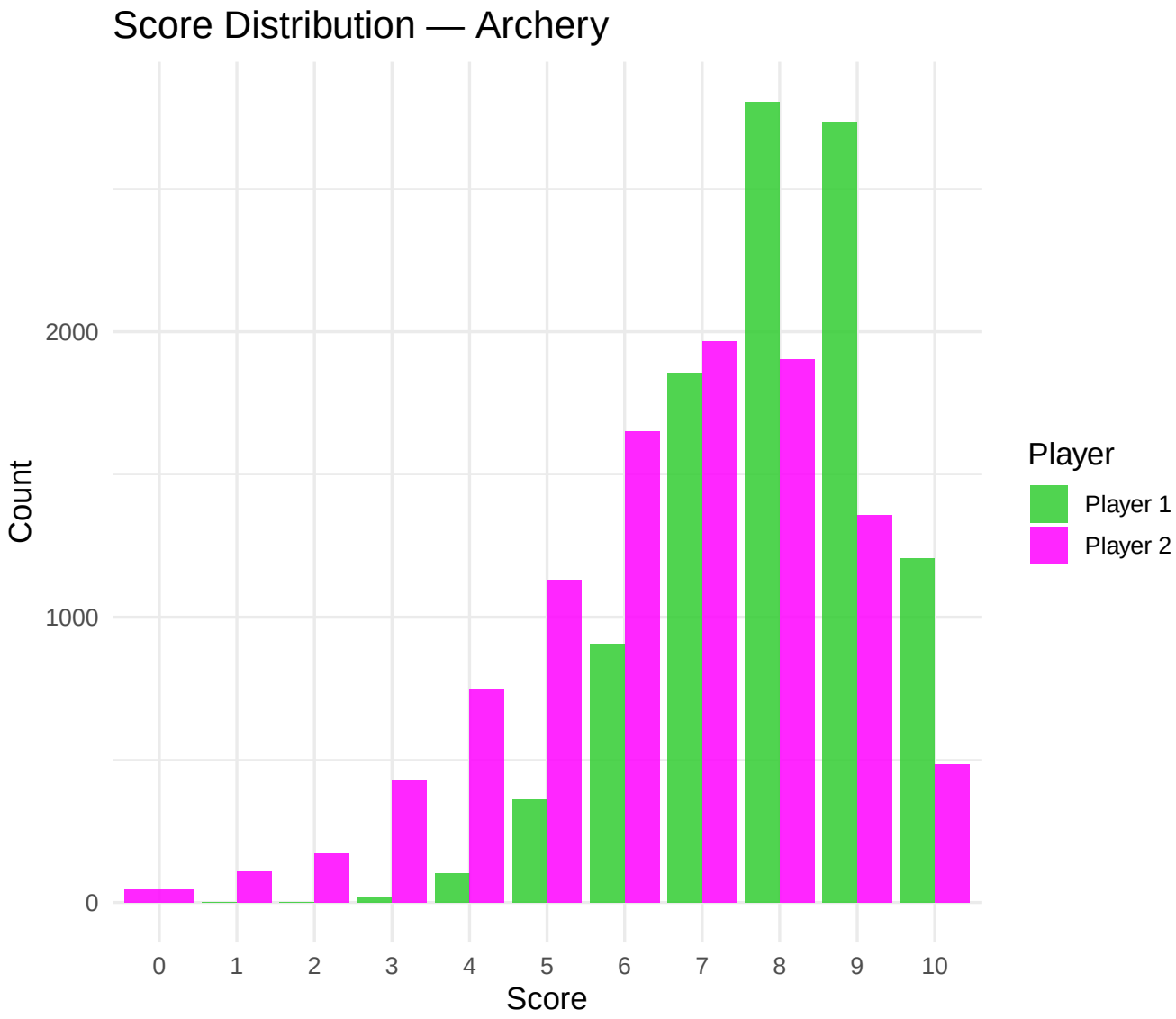


Figure 2: Score frequency distribution for both players.

Head-to-Head Probabilities (Single Arrow)

```
p1_beats_p2 <- mean(shots1$score > shots2$score)
p2_beats_p1 <- mean(shots2$score > shots1$score)
p_draw      <- mean(shots1$score == shots2$score)

cat("P(Player 1 score > Player 2 score):", round(p1_beats_p2, 4), "\n")
```

```
## P(Player 1 score > Player 2 score): 0.6259
```

```
cat("P(Player 2 score > Player 1 score):", round(p2_beats_p1, 4), "\n")
```

```
## P(Player 2 score > Player 1 score): 0.2272
```

```
cat("P(Draw):                           ", round(p_draw, 4), "\n")
```

```
## P(Draw):                           0.1469
```

To examine how the two players perform in direct competition, the simulation estimated the head-to-head probabilities for a single-arrow contest. By comparing 10,000 paired scores, it calculated the likelihood of each possible outcome. The results strongly favour Player 1's high-precision profile. They outscored Player 2 in 62.59% of cases, while Player 2 achieved a win rate of just 22.72%. The remaining 14.69% of simulations resulted in a draw. These findings are consistent with Player 1's lower variability and higher average score. As such, they provide a solid baseline for extending the model to full multi-arrow matches under official federation rules. —

Part 2: World Archery Federation — Individual Set-Play

Rules (12.1.4.1 — Individual Set-Play)

Under World Archery Federation (WAF) rules for Individual Set-Play:

- Each **set** consists of **3 arrows** per archer.
- The archer with the higher set total scores **2 set points**; a tie gives each archer **1 set point**.
- The first archer to reach **6 set points** wins the match.
- If tied at 5–5, a **shoot-off** (1 arrow) decides the match (closest to centre wins).

```
# Simulate a single set (3 arrows each)
# Returns set points awarded: c(sp1, sp2)
simulate_set <- function(mu1, Sigma1, mu2, Sigma2) {
  s1 <- sum(simulate_archer(3, mu1, Sigma1)$score)
  s2 <- sum(simulate_archer(3, mu2, Sigma2)$score)
  if (s1 > s2)      c(2L, 0L)
  else if (s2 > s1) c(0L, 2L)
  else              c(1L, 1L)
}

# Simulate shoot-off: 1 arrow, closest to centre wins (smaller r wins)
```

```r
simulate_shootoff <- function(mu1, Sigma1, mu2, Sigma2) {
  r1 <- simulate_archer(1, mu1, Sigma1)$r
  r2 <- simulate_archer(1, mu2, Sigma2)$r
  if (r1 < r2) 1L else if (r2 < r1) 2L else sample(1:2, 1)  # true tie: coin flip
}

# Simulate a full WAF match
simulate_match <- function(mu1, Sigma1, mu2, Sigma2) {
  sp <- c(0L, 0L)
  repeat {
    sp <- sp + simulate_set(mu1, Sigma1, mu2, Sigma2)
    # Check win condition
    if (sp[1] >= 6 && sp[1] > sp[2]) return(1L)
    if (sp[2] >= 6 && sp[2] > sp[1]) return(2L)
    # Shoot-off at 5-5
    if (sp[1] >= 5 && sp[2] >= 5) return(simulate_shootoff(mu1, Sigma1, mu2, Sigma2))
  }
}
```

Simulate Many Matches

```r
M <- 5000  # number of matches

winners <- replicate(M, simulate_match(mu1, Sigma1, mu2, Sigma2))

p1_wins <- mean(winners == 1)
ci_match <- mc_ci(as.integer(winners == 1))

cat("P(Player 1 wins match):", round(p1_wins, 4), "\n")
```

```
## P(Player 1 wins match): 0.9656
```

```r
cat("95% MC CI: [", round(ci_match["lower"], 4), ",",
    round(ci_match["upper"], 4), "]\n")
```

```
## 95% MC CI: [ 0.9605 , 0.9707 ]
```

To assess performance under realistic competitive conditions, the stochastic model was extended to simulate full matches using the official World Archery Federation (WAF) individual set-play format. Unlike a simple cumulative scoring system, set-play introduces a level of strategic structure. Each set consists of three arrows per archer, with two set points awarded to the player achieving the higher total score, and one point each in the event of a tie. The match is won by the first player to reach six set points. The model also accounts for the important edge case of a 5–5 tie. In this situation, a sudden-death shoot-off is triggered. Here, the standard scoring rings are ignored, and each player takes a single arrow. The winner is determined solely by which arrow lands closest to the centre, based on the continuous radial distance r. Using this framework, a Monte Carlo simulation of 5,000 independent matches was carried out to estimate match-winning probabilities. The results clearly demonstrate how small statistical advantages accumulate over repeated events. While Player 1 had around a 62.6% chance of winning a single-arrow contest, their overall match win probability rises sharply to 96.56% under the set-play format. This estimate is highly reliable, with a tight 95% confidence

interval of [0.9605, 0.9707]. This substantial increase highlights a key statistical feature of the format: set-play acts as a natural filter against randomness. By grouping shots into sets and resetting scores between them, the system reduces the impact of occasional poor shots. As a result, Player 2's higher variability and slight aiming offset are consistently penalised, making an upset over a full match extremely unlikely.

```
cum_wins <- cumsum(winners == 1) / seq_along(winners)

ggplot(data.frame(match = 1:M, prob = cum_wins), aes(x = match, y = prob)) +
  geom_line(color = "firebrick", linewidth = 0.7) +
  geom_hline(yintercept = p1_wins, linetype = "dashed", color = "black") +
  geom_hline(yintercept = ci_match["lower"], linetype = "dotted", color = "grey50") +
  geom_hline(yintercept = ci_match["upper"], linetype = "dotted", color = "grey50") +
  labs(title  = "Convergence of Player 1 Win Probability (WAF Set-Play)",
       x = "Number of simulated matches",
       y = "Cumulative P(Player 1 wins)") +
  theme_minimal(base_size = 13)
```

To confirm that the chosen sample size was sufficient for the World Archery Federation match simulations, a convergence plot was produced to track Player 1's cumulative win probability over time. This provides a clear way of assessing whether the Monte Carlo simulation ran for long enough to generate stable and reliable results. As expected under the Law of Large Numbers, the early stages of the simulation show considerable fluctuation. Within the first 1,000 matches, the rolling average varies noticeably, as individual outcomes still have a strong influence on the overall estimate. However, as the number of simulated matches increases towards 5,000, this variability gradually diminishes. The cumulative probability steadily stabilises, converging closely on the final estimated match win probability (indicated by the central dashed line) and remaining well within the 95% Monte Carlo confidence bounds (shown by the dotted grey lines). This behaviour demonstrates that the simulation has reached statistical stability. It confirms that 5,000 iterations are sufficient to capture the true underlying difference in performance between the two players, without being distorted by early random variation.

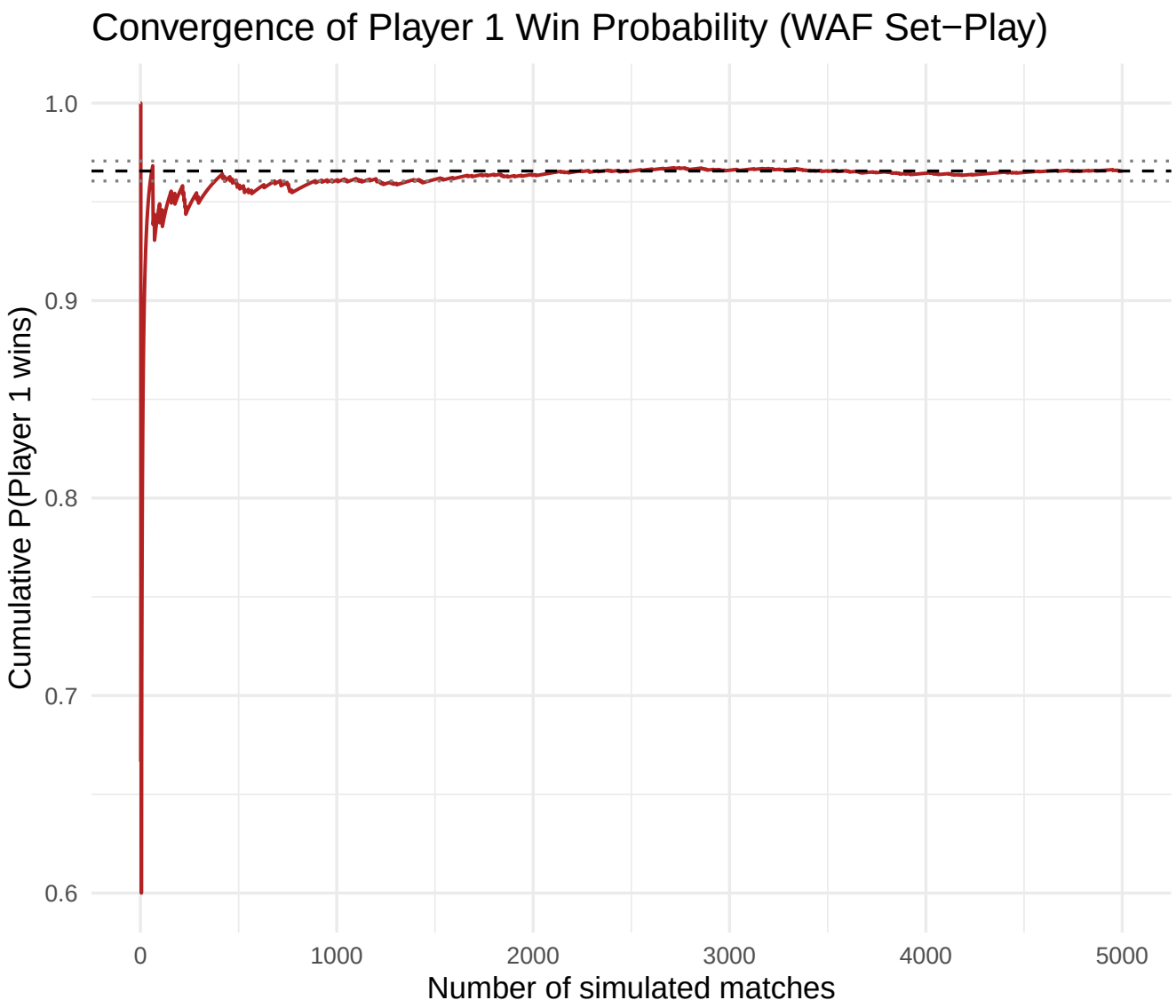


Figure 3: Estimated win probability for Player 1 across simulated matches with 95% Monte Carlo CI.

Part 3: Parameter Sensitivity — What Makes a Better Archer?

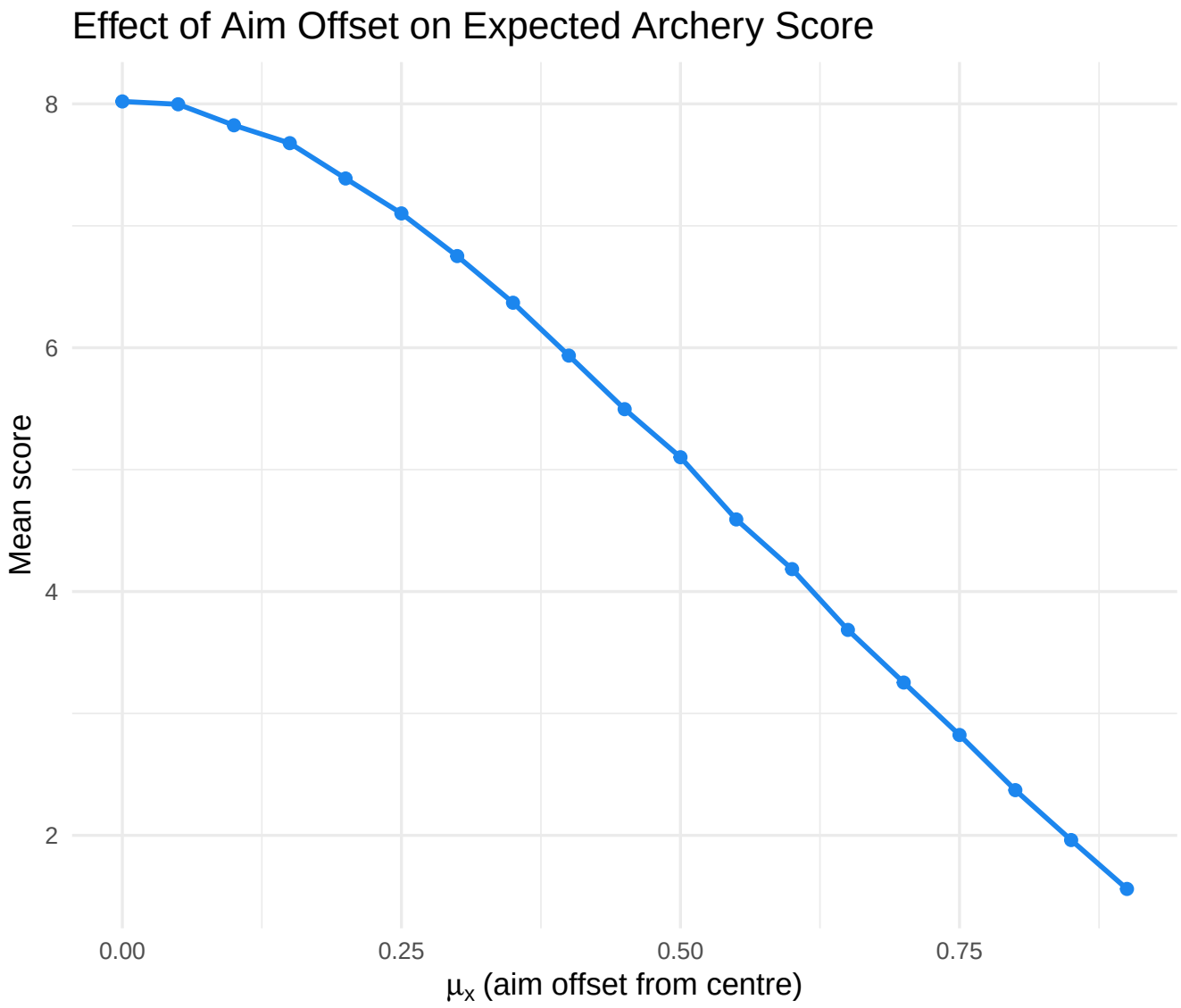
We vary individual parameters of the bivariate normal distribution to understand their influence on expected score.

```
sigma_vals <- seq(0.02, 0.30, by = 0.02)

mean_scores <- sapply(sigma_vals, function(s) {
  Sigma_test <- matrix(c(s^2, 0, 0, s^2), 2, 2)
  mean(simulate_archer(5000, c(0, 0), Sigma_test)$score)
})

ggplot(data.frame(sigma = sigma_vals, mean_score = mean_scores),
       aes(x = sigma, y = mean_score)) +
  geom_line(color = "forestgreen", linewidth = 1) +
  geom_point(color = "forestgreen", size = 2) +
  labs(title = "Effect of Standard Deviation on Expected Archery Score",
       x = expression(sigma ~ "(equal in both directions)"),
       y = "Mean score") +
  theme_minimal(base_size = 13)
```

To quantify how an archer's physical consistency affects overall performance, a sensitivity analysis was carried out to isolate the impact of stochastic spread, represented by the standard deviation σ, on the expected mean score. In this controlled setup, the aiming point μ was fixed precisely at the Fcentre (0,0), and covariance was set to zero. This ensured that any change in performance was driven solely by variability in shot placement, rather than systematic aiming error. The model tested a range of standard deviation values from 0.02 to 0.30. At each step, 5,000 arrows were simulated to produce a stable estimate of the expected score. The results show a clear and steadily decreasing relationship between variability and scoring performance. At very low standard deviations ($\sigma \approx 0.02$), the simulated archer performs almost perfectly, with shots tightly grouped in the centre and an expected score approaching 10. As σ increases, however, the expected score drops sharply. This decline occurs because a wider spread in the bivariate normal distribution pushes more shots away from the high-scoring centre and into the lower-value outer rings. The steepness of this curve highlights an important point: even small reductions in consistency can lead to substantial drops in expected score, even when aim remains perfectly centred.

```
# Effect of aiming off-centre
offset_vals <- seq(0, 0.9, by = 0.05)
Sigma_fixed  <- matrix(c(0.04, 0, 0, 0.04), 2, 2)
```

```r
mean_scores_aim <- sapply(offset_vals, function(d) {
  mean(simulate_archer(5000, c(d, 0), Sigma_fixed)$score)
})

ggplot(data.frame(offset = offset_vals, mean_score = mean_scores_aim),
       aes(x = offset, y = mean_score)) +
  geom_line(color = "dodgerblue2", linewidth = 1) +
  geom_point(color = "dodgerblue2", size = 2) +
  labs(title = "Effect of Aim Offset on Expected Archery Score",
       x = expression(mu[x] ~ "(aim offset from centre)"),
       y = "Mean score") +
  theme_minimal(base_size = 13)
```



Following the analysis of physical variability, a second sensitivity analysis was carried out to isolate the effect of systematic aiming error on performance. This was designed to represent real-world influences such as wind drift or slight misalignment of the bow sight. In this case, the spread of the distribution was held constant using a fixed covariance matrix with a variance of 0.04 in both the horizontal and vertical

directions. The horizontal aiming point, μx, was then shifted gradually from the centre of the target (0.0) towards the edge (0.9) in increments of 0.05. At each point, 5,000 Monte Carlo iterations were run to produce a stable estimate of the expected score. The resulting graph shows a clear, non-linear decline in performance as the aiming offset increases. When the offset is very small, such as between 0.0 and 0.1, the effect on the mean score is minimal because most of the probability mass still lies within the higher-scoring central rings. However, as μx moves further away from the origin, the centre of the distribution shifts across the lower-scoring regions of the target. This produces a steep drop in expected score. By the time the aim point reaches 0.9, close to the outer boundary, the mean score falls to below 2.0. At this level of offset, the central rings are no longer accessible to most of the distribution, and a sizeable proportion of arrows fall outside the target altogether, resulting in zero scores. Overall, this confirms that while small aiming errors may be tolerated, larger offsets have a severe impact on expected performance. —

Part 4: Darts Simulation

Dartboard Specification

A standard dartboard has:

- **Sectors**: 20 numbered sectors (1–20), each spanning 18°, arranged in the standard order.
- **Single** (main sector area), **Double** (outer narrow ring, ×2), **Triple** (inner narrow ring, ×3).
- **Outer Bullseye** (score 25), **Inner Bullseye / Bull** (score 50).

We normalise the dartboard so the double ring's outer edge has radius = 1.

```
# Standard sector order (clockwise from top, i.e., starting at -9 degrees)
sector_order <- c(20, 1, 18, 4, 13, 6, 10, 15, 2, 17, 3, 19, 7, 16, 8, 11, 14, 9, 12, 5)

# Normalised radii (outer edge of each zone, relative to double outer = 1)
r_bull_inner  <- 0.034   # Inner bull (Bull's-eye, score 50)
r_bull_outer  <- 0.080   # Outer bull (score 25)
r_triple_in   <- 0.535   # Inner edge of triple ring
r_triple_out  <- 0.580   # Outer edge of triple ring
r_double_in   <- 0.935   # Inner edge of double ring
r_double_out  <- 1.000   # Outer edge of double ring (boundary)

# Assign score from (r, phi) - phi in degrees [0, 360)
darts_score <- function(r, phi) {
  # Bullseyes
  if (r <= r_bull_inner) return(50L)
  if (r <= r_bull_outer) return(25L)
  # Outside board
  if (r > r_double_out)  return(0L)

  # Determine sector number
  # Convert polar phi (counter-clockwise from 3 o'clock)
  # to a clockface theta (clockwise from 12 o'clock)
  theta_clock <- (90 - phi) %% 360

  # Sectors are 18 deg each. Shift by 9 degrees so Sector 20 sits cleanly across 0.
  theta_rot  <- (theta_clock + 9) %% 360
  idx        <- floor(theta_rot / 18) + 1  # 1-indexed sector
  sector     <- sector_order[idx]
```

```r
  # Determine multiplier
  if (r <= r_triple_in)  return(as.integer(sector))          # single (inner)
  if (r <= r_triple_out) return(as.integer(sector * 3))      # triple
  if (r <= r_double_in)  return(as.integer(sector))          # single (outer)
  return(as.integer(sector * 2))                             # double
}

# Vectorised wrapper
darts_score_vec <- function(r_vec, phi_vec) {
  mapply(darts_score, r_vec, phi_vec)
}
```

The dartboard scoring function must handle two independent dimensions simultaneously: the radial zone and the angular sector. This makes it fundamentally more complex than the archery scoring function, which required only the radius r.

Radially, the board is divided into five distinct zones: inner bullseye (score 50), outer bullseye (score 25), the single area between the outer bull and the triple ring, the triple ring itself (where the face value is multiplied by three), the single area between the triple and double rings, and finally the double ring (face value multiplied by two). Any dart landing beyond the outer edge of the double ring scores zero.

Angularly, the 20 sectors are aligned in the standard clockwise arrangement used in professional play. Critically, sector 20 is centred at the top of the board, which in our Cartesian coordinate system corresponds to 0° (the positive x-axis, as the board is represented in the xy-plane). To ensure that sector 20 is symmetrically straddled around 0° rather than starting at 0°, the raw angular coordinate ϕ is rotated by 9° before sector assignment. This rotation correctly places the sector boundaries at 9°, 27°, 45°, and so on, matching the physical geometry of a real board.

The scoring logic was verified to handle all boundary cases cleanly, including the bullseye zones and shots landing precisely on the boundary between the treble ring and the adjacent single areas.

Simulate Darts Throws

```r
simulate_darts <- function(N, mu, Sigma) {
  shots <- rmvnorm(N, mean = mu, sigma = Sigma)
  polar <- to_polar(shots[, 1], shots[, 2])
  scores <- darts_score_vec(polar$r, polar$phi)
  data.frame(x = shots[, 1], y = shots[, 2],
             r = polar$r, phi = polar$phi, score = scores)
}

# Darts player aiming at treble 20 (top of board)
# Treble 20 is at phi ~ 0 deg, r ~ 0.557 (midpoint of triple ring)
mu_d1    <- c(0, 0.557)   # aim at top (treble 20)
Sigma_d1 <- matrix(c(0.15^2, 0.00,
                     0.00, 0.15^2), nrow = 2)

darts1 <- simulate_darts(N, mu_d1, Sigma_d1)
cat("Darts Player 1 - Mean score per dart:", round(mean(darts1$score), 3), "\n")
```

```
## Darts Player 1 - Mean score per dart: 14.196
```

```r
cat("Most common score:", as.integer(names(which.max(table(darts1$score)))), "\n")
```

```
## Most common score: 20
```

To ensure the simulation reflects realistic human performance, the player's physical consistency must be grounded in real-world measurements. On a standard tournament dartboard, the radial distance from the centre of the bullseye to the outer edge of the double wire is exactly 170mm. Because our mathematical model normalises this outer boundary to a radius of 1.0, any physical standard deviation must be scaled proportionally.

Empirical studies of dart-throwing biomechanics suggest that a strong amateur (a pub league player) exhibits a physical standard deviation of approximately 25mm. Scaling this to our normalised board yields a standard deviation of $\sigma \approx 25/170 \approx 0.147$. For the baseline player profile in this simulation, we round this to σ=0.15. The covariance matrix is therefore constructed using a variance of 0.15^2 in both the horizontal and vertical directions. This represents a capable player who scores reasonably well but possesses enough variance to frequently drift into neighbouring penalty sectors, perfectly capturing the risk-reward dynamic of targeting Treble 20.

With a mean score per dart in the region of 14 and a modal score of 20, this profile captures the characteristic risk-reward trade-off inherent in aiming at Treble 20. The majority of darts land in the single 20 region, delivering a moderate average score. However, the narrow treble band means that slight misses to the left or right drift into the adjacent sectors — sector 1 and sector 5 — which carry much lower face values of 1 and 5 respectively, and can significantly reduce the expected return for less accurate players.

Visualise Darts Shot Distribution

```r
ggplot(darts1 %>% sample_n(min(500, nrow(darts1))),
       aes(x = x, y = y, color = factor(pmin(score, 60)))) +
  geom_point(alpha = 0.8, size = 1) +

  scale_color_viridis_d(option = "turbo") +

  annotate("path", color = "firebrick",
           x = r_bull_inner * cos(seq(0, 2*pi, l=200)),
           y = r_bull_inner * sin(seq(0, 2*pi, l=200))) +
  annotate("path", color = "forestgreen",
           x = r_bull_outer * cos(seq(0, 2*pi, l=200)),
           y = r_bull_outer * sin(seq(0, 2*pi, l=200))) +

  # Inner AND Outer edges of the TRIPLE ring
  annotate("path", color = "dodgerblue2",
           x = r_triple_in * cos(seq(0, 2*pi, l=200)),
           y = r_triple_in * sin(seq(0, 2*pi, l=200))) +
  annotate("path", color = "dodgerblue2",
           x = r_triple_out * cos(seq(0, 2*pi, l=200)),
           y = r_triple_out * sin(seq(0, 2*pi, l=200))) +

  # Inner AND Outer edges of the DOUBLE ring
  annotate("path", color = "black",
           x = r_double_in * cos(seq(0, 2*pi, l=200)),
           y = r_double_in * sin(seq(0, 2*pi, l=200))) +
  annotate("path", color = "black",
```

```
          x = r_double_out * cos(seq(0, 2*pi, l=200)),
          y = r_double_out * sin(seq(0, 2*pi, l=200))) +

  # The 20 radial lines dividing the sectors
  annotate("segment",
          x = r_bull_outer * cos(seq(9, 351, by = 18) * pi / 180),
          y = r_bull_outer * sin(seq(9, 351, by = 18) * pi / 180),
          xend = r_double_out * cos(seq(9, 351, by = 18) * pi / 180),
          yend = r_double_out * sin(seq(9, 351, by = 18) * pi / 180),
          color = "black", alpha = 0.5, linewidth = 0.3) +

  coord_fixed() +

  labs(title = "Dart Throws (Aiming at Treble 20)",
       color = "Score Hit", x = "x", y = "y") +

  guides(colour = guide_legend(override.aes = list(size = 4))) +

  theme_minimal(base_size = 13)
```

The scatter plot maps each simulated dart throw onto the Cartesian plane, overlaid with a schematic representation of the key scoring boundaries. The innermost red circle marks the inner bullseye, the green circle the outer bullseye, the blue rings delimit the triple band, and the black rings mark the double ring and outer board boundary. The 20 radial lines dividing the sectors are also shown, providing spatial context for the angular arrangement of the board.

The cluster of points is clearly centred near the top of the board at the Treble 20 position, consistent with the bivariate normal aim point specified at $(0, 0.557)$. The colour gradient, running from deep violet for low scores through to deep red for high scores, makes the scoring geography immediately visible: the highest-scoring throws are tightly concentrated in the narrow treble band, whilst the broader scatter around it falls predominantly into the adjacent single 20 region.

The plot also reveals the asymmetric cost of missing to either side. Drifts towards the left fall into sector 5 (score 5), whilst drifts to the right land in sector 1 (score 1). Both neighbours are substantially worse than sector 19 (to the left of treble 20 when approached from the centre), which is one reason why many coaches advocate aiming at Treble 19 rather than Treble 20 for players whose variance is non-trivial — a strategic question addressed directly in Part 6.

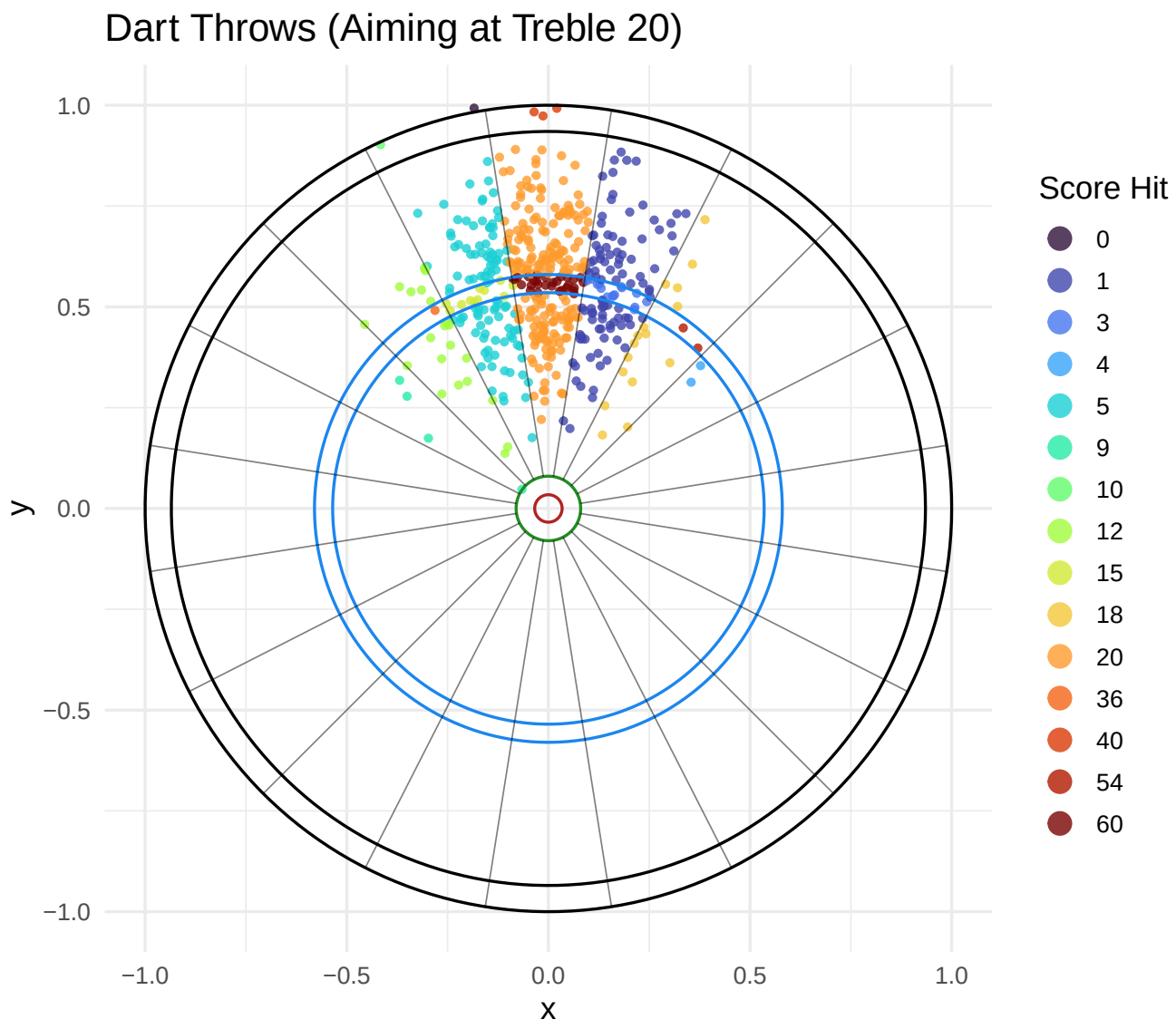


Figure 4: Simulated dart throws for a player aiming at Treble 20.

Simulating a Game of 301 (Double Out)

In 301, each player starts at 301 points and must reach exactly 0. The final dart must land on a **double** field. If a throw would reduce the score below 0 or to 1, those darts in that turn are busted.

Strategy assumptions: - While score $>$ 40: aim at **treble 20**. - While 41 score 40 (finishing range): aim at the **appropriate double**. - Score 40: aim at the appropriate double (if even), or aim at Single 1 (if odd) to restore an even remaining score.

```
# Return the aim point for a given remaining score
aim_for_score <- function(remaining) {

  if (remaining > 40) {
    # Aim at Treble 20
    # Sector 20 is at the top of the board (90 degrees / x=0, y=0.557)
    return(list(mu = c(0, 0.557), Sigma = matrix(c(0.15^2, 0, 0, 0.15^2), 2)))

  } else if (remaining %% 2 != 0) {
    # If ODD, aim at Single 1 to leave an even number
    # Sector 1 is index 1. Polar angle: 90 - (1 * 18) = 72 degrees
    angle_rad <- 72 * pi / 180
    mu_aim <- c(0.75 * cos(angle_rad), 0.75 * sin(angle_rad))
    return(list(mu = mu_aim, Sigma = matrix(c(0.15^2, 0, 0, 0.15^2), 2)))

  } else {
    # If EVEN, aim at the exact double to win
    target_double <- remaining / 2
    idx <- which(sector_order == target_double) - 1

    # Correctly map the sector index to a physical polar angle
    polar_deg <- 90 - (idx * 18)
    angle_rad <- polar_deg * pi / 180
    r_aim <- (r_double_in + r_double_out) / 2

    mu_aim <- c(r_aim * cos(angle_rad), r_aim * sin(angle_rad))
    return(list(mu = mu_aim, Sigma = matrix(c(0.15^2, 0, 0, 0.15^2), 2)))
  }
}

# Simulate one game of 301, double out; returns number of darts thrown
simulate_301 <- function(mu_default, Sigma_default, max_darts = 500) {
  score  <- 301L
  darts_thrown <- 0L

  while (score > 0 && darts_thrown < max_darts) {
    aim   <- aim_for_score(score)
    throw <- simulate_darts(1, aim$mu, Sigma_default)
    darts_thrown <- darts_thrown + 1L
    pts <- throw$score

    # Check for bust
    new_score <- score - pts
    if (new_score < 0 || new_score == 1) next   # bust - don't update score
    if (new_score == 0) {
      # Must finish on a double
```

```
      r_hit  <- throw$r
      in_double <- r_hit >= r_double_in & r_hit <= r_double_out
      if (in_double) { score <- 0L } # valid finish
      # else bust (do nothing)
    } else {
      score <- new_score
    }
  }
  darts_thrown
}

# Simulate many games
G <- 2000
game_lengths <- replicate(G, simulate_301(mu_d1, Sigma_d1))

cat("Mean darts per game:", round(mean(game_lengths), 2), "\n")
```

```
## Mean darts per game: 30.06
```

```
cat("Min darts:          ", min(game_lengths), "\n")
```

```
## Min darts:           11
```

```
cat("Max darts:          ", max(game_lengths), "\n")
```

```
## Max darts:           83
```

```
mean_d <- mean(game_lengths)

ggplot(data.frame(darts = game_lengths), aes(x = darts)) +
  # boundary = 0 ensures the bins align perfectly with groups of 3 darts
  geom_histogram(binwidth = 3, boundary = 0, fill = "darkorange", color = "white", alpha = 0.85) +

  geom_vline(xintercept = mean_d, linetype = "dashed", color = "dodgerblue2", linewidth = 1) +

  # y = 350 assumes max frequency is around there
  annotate("text", x = mean_d + 1.5, y = 350,
           label = paste("Mean:\n", round(mean_d, 1), "Darts"),
           hjust = 0, color = "dodgerblue4", fontface = "bold") +

  labs(title = "Distribution of Game Length in 301 (Double Out)",
       x = "Number of darts thrown",
       y = "Frequency") +
  theme_minimal(base_size = 13)
```

The 301 double-out simulation captures the full complexity of a competitive darts leg: maximising scoring output during the opening phase, correctly managing odd remainders that would otherwise prevent a double finish, and successfully converting a finishing double under pressure. The finishing strategy implemented here mirrors standard professional practice: whilst the remaining score exceeds 40, the player targets Treble 20 to reduce the score as quickly as possible. Once inside the finishing range, the strategy switches to targeting the appropriate double, or Single 1 when the remaining score is odd in order to restore an even number.

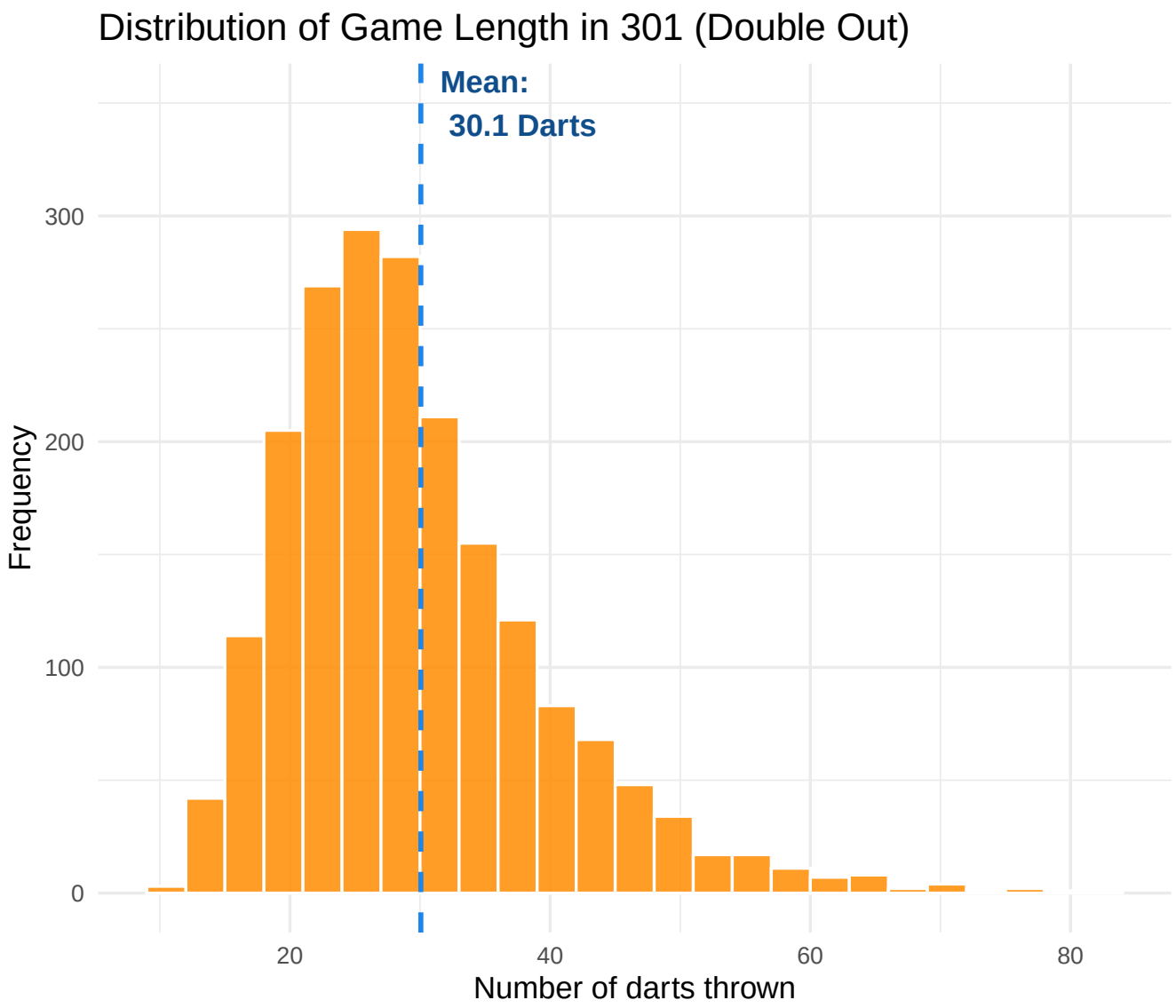


Figure 5: Distribution of number of darts required to finish a game of 301.

The histogram of game lengths across 2,000 simulated legs displays a right-skewed distribution, as one would expect from a process with a theoretical minimum but no hard upper bound. Notably, a bin width of 3 was deliberately chosen for this visualisation to reflect the physical structure of the sport: a player throws 3 darts per turn (a "visit" to the oche). By grouping the data in this way, the histogram effectively visualises the number of turns required to finish the leg rather than just raw dart counts.

The majority of games are completed within a moderate range of darts, with the distribution peaking near the mean value shown by the dashed blue line. The long right tail captures legs in which the player experienced repeated busts on finishing attempts or endured a string of single scores during the high-score phase.

This distributional shape has practical implications. Because the variance in game length is substantial, a single leg is a poor basis for inferring a player's true skill level. However, averaged across many legs, the mean game length provides a stable and informative summary statistic of overall performance quality, particularly double-conversion efficiency, which is the dominant source of variability in the finishing phase.

Part 6: Optimal Targeting — Expected Score by Inaccuracy Level

While simulating full games of 301 provides a realistic view of match lengths, the strict rules of the game—particularly the "double-out" requirement—introduce significant statistical noise. At higher variance levels, players can become trapped at the end of a leg, spending dozens of darts attempting to hit a specific double. This finishing bottleneck heavily dilutes the measurable impact of their mid-game scoring strategy.

To rigorously determine the mathematically optimal target on the dartboard, we must strip away these complex game mechanics and isolate a single dart throw. In this section, we transition from full-game simulations to a pure Expected Value (EV) framework.

By simulating thousands of independent darts per target across a continuous spectrum of player inaccuracy (Σ), we can map the true strategic topography of the dartboard. This approach bypasses the noise of the double-out trap and focuses purely on spatial probability, revealing exactly when a player's physical variance dictates that they should abandon Treble 20 in favour of mathematically safer alternatives.

STRATEGY COMPARISON: Expected Score per Dart

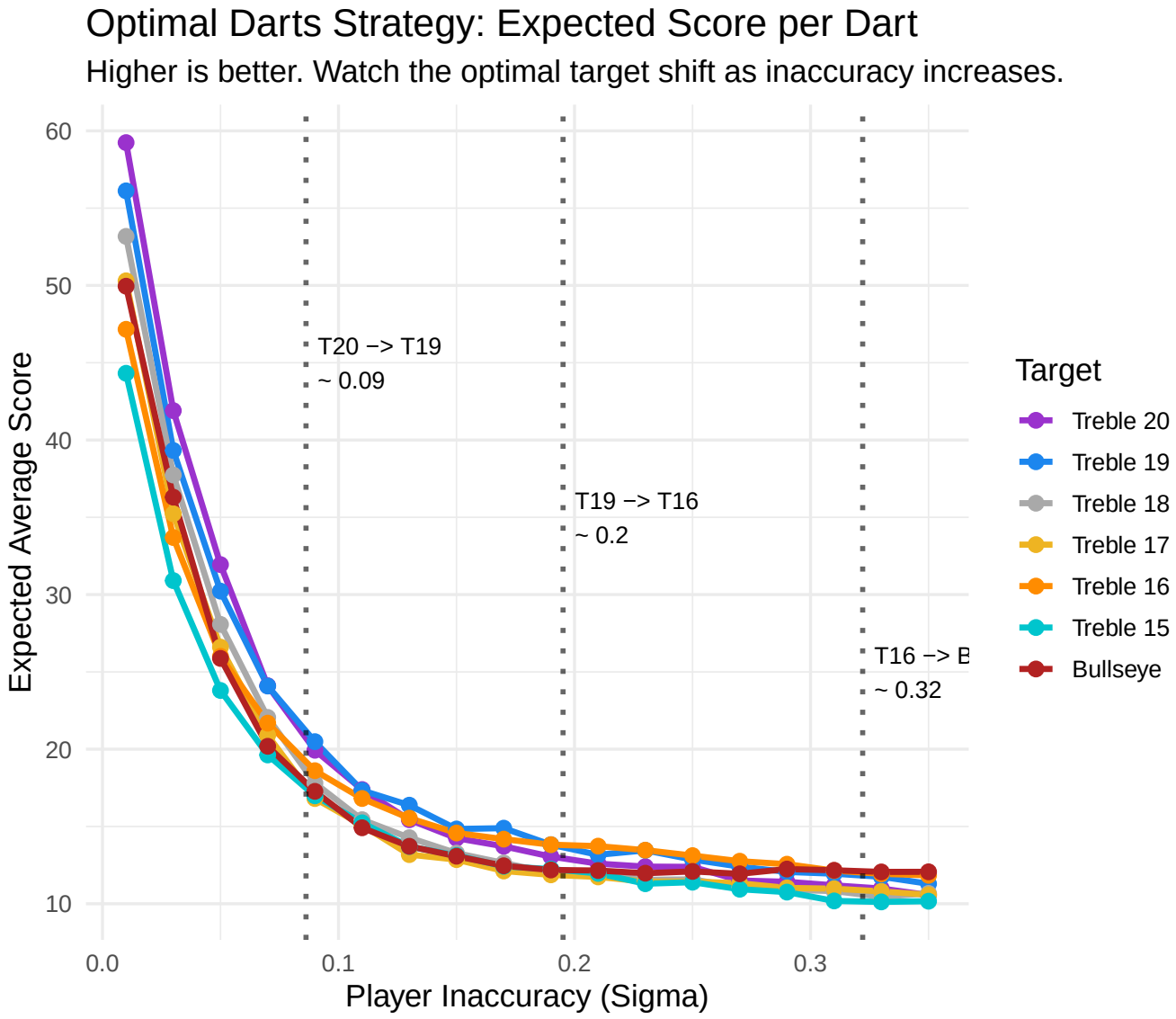
```
# 1. Define the exact coordinates for our targets
r_aim_treble <- 0.557

# Helper function to find the exact (x,y) coordinates for any Treble
get_treble_aim <- function(target_sector) {
  idx <- which(sector_order == target_sector) - 1
  polar_deg <- 90 - (idx * 18) # Maps clockface to standard polar angle
  angle_rad <- polar_deg * pi / 180
  c(r_aim_treble * cos(angle_rad), r_aim_treble * sin(angle_rad))
}

aim_points <- list(
  "Treble 20" = get_treble_aim(20),
  "Treble 19" = get_treble_aim(19),
  "Treble 18" = get_treble_aim(18),
  "Treble 17" = get_treble_aim(17),
  "Treble 16" = get_treble_aim(16),
  "Treble 15" = get_treble_aim(15),
```

```
  "Bullseye"  = c(0, 0)
)

# 2. Setup the simulation parameters
sigma_vals <- seq(0.01, 0.35, by = 0.02)
N_darts <- 5000
simple_results <- data.frame()

cat("Running Expected Value Simulation...\n")
```

```
## Running Expected Value Simulation...
```

```
# 3. The Simple Simulation Loop
for (s in sigma_vals) {
  current_Sigma <- matrix(c(s^2, 0, 0, s^2), 2, 2)

  for (target_name in names(aim_points)) {
    mu <- aim_points[[target_name]]

    throws <- simulate_darts(N_darts, mu, current_Sigma)
    expected_score <- mean(throws$score)

    simple_results <- rbind(simple_results, data.frame(
      Sigma = s,
      Target = factor(target_name, levels = c("Treble 20", "Treble 19", "Treble 18", "Treble 17",
                                              "Treble 16", "Treble 15", "Bullseye")),
      Expected_Score = expected_score
    ))
  }
}

# ======================================================================
# DYNAMIC CROSSOVER CALCULATION
# ======================================================================

smooth_t20  <- loess(Expected_Score ~ Sigma, data = subset(simple_results, Target == "Treble 20"),
                     span = 0.65)
smooth_t19  <- loess(Expected_Score ~ Sigma, data = subset(simple_results, Target == "Treble 19"),
                     span = 0.65)
smooth_t16  <- loess(Expected_Score ~ Sigma, data = subset(simple_results, Target == "Treble 16"),
                     span = 0.65)
smooth_bull <- loess(Expected_Score ~ Sigma, data = subset(simple_results, Target == "Bullseye"),
                     span = 0.65)

fine_grid <- seq(0.01, 0.35, length.out = 1000)

p_t20  <- predict(smooth_t20, newdata = data.frame(Sigma = fine_grid))
p_t19  <- predict(smooth_t19, newdata = data.frame(Sigma = fine_grid))
p_t16  <- predict(smooth_t16, newdata = data.frame(Sigma = fine_grid))
p_bull <- predict(smooth_bull, newdata = data.frame(Sigma = fine_grid))

t19_takes_lead <- fine_grid[which(p_t19 > p_t20)[1]]
t16_takes_lead <- fine_grid[which(p_t16 > p_t19 & p_t16 > p_t20)[1]]
```

```r
bull_takes_lead <- fine_grid[which(p_bull > p_t16 & p_bull > p_t19 & p_bull > p_t20)[1]]

cross_20_19   <- ifelse(is.na(t19_takes_lead), 0.08, t19_takes_lead)
cross_19_16   <- ifelse(is.na(t16_takes_lead), 0.18, t16_takes_lead)
cross_16_bull <- ifelse(is.na(bull_takes_lead), 0.30, bull_takes_lead)

# ======================================================================
# PLOTTING THE EXPECTED VALUE CURVES
# ======================================================================

ggplot(simple_results, aes(x = Sigma, y = Expected_Score, color = Target)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 2.5) +
  scale_color_manual(values = c(
    "Treble 20" = "darkorchid3",
    "Treble 19" = "dodgerblue2",
    "Treble 18" = "darkgray",
    "Treble 17" = "goldenrod2",
    "Treble 16" = "darkorange",
    "Treble 15" = "turquoise3",
    "Bullseye"  = "firebrick"
  )) +

  geom_vline(xintercept = cross_20_19, linetype = "dotted", color = "black", alpha = 0.6,
             linewidth = 1) +
  geom_vline(xintercept = cross_19_16, linetype = "dotted", color = "black", alpha = 0.6,
             linewidth = 1) +
  geom_vline(xintercept = cross_16_bull, linetype = "dotted", color = "black", alpha = 0.6,
             linewidth = 1) +

  annotate("text", x = cross_20_19 + 0.005, y = 45,
           label = paste("T20 -> T19\n~", round(cross_20_19, 2)), hjust = 0, size = 3.5) +
  annotate("text", x = cross_19_16 + 0.005, y = 35,
           label = paste("T19 -> T16\n~", round(cross_19_16, 2)), hjust = 0, size = 3.5) +
  annotate("text", x = cross_16_bull + 0.005, y = 25,
           label = paste("T16 -> Bull\n~", round(cross_16_bull, 2)), hjust = 0, size = 3.5) +

  labs(title = "Optimal Darts Strategy: Expected Score per Dart",
       subtitle = "Higher is better. Watch the optimal target shift as inaccuracy increases.",
       x = "Player Inaccuracy (Sigma)",
       y = "Expected Average Score") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "right")
```



The expected value chart is one of the most practically useful outputs of this entire simulation study. It reveals how the optimal primary target in darts depends critically on a player's level of inaccuracy, as measured by σ.

At very low values of σ — corresponding to elite professional players — Treble 20 yields the highest expected score per dart, as the vast majority of throws land within or very close to the treble band, and the risk of drifting into the penalty neighbours (sectors 1 and 5) is negligible. The first crossover point marks the level of inaccuracy at which Treble 19 overtakes Treble 20 as the superior strategy. Beyond this point, the more forgiving neighbourhood of Treble 19 (flanked by 3 and 7) means that misses are less costly on average.

As inaccuracy continues to increase, a second crossover occurs where Treble 16 becomes the optimal target. Treble 16 is flanked by sectors 8 and 11, which are more valuable than Treble 19's neighbours at higher variance levels. Finally, at the highest inaccuracy levels tested, the bullseye emerges as the optimal aim point. This is counterintuitive at first glance — the bullseye scores only 25 for the outer ring and 50 for the inner — but when σ is very large, a player cannot reliably target any specific sector. Aiming at the geometric centre of the board maximises the average score because the shot distribution is then centred over the highest-density region of the board, with misses distributed symmetrically across all sectors rather than being biased towards low-scoring neighbours.

These crossover thresholds provide an empirical, data-driven basis for coaching advice. Rather than defaulting to Treble 20 as a universal rule, a coach could measure a player's σ during practice sessions and use this chart to identify the statistically optimal primary target for that individual's current skill level.

Part 5: Live In-Play Odds Simulator

A natural extension of the Monte Carlo framework is to apply it in real time during a match, recalculating win probabilities after every turn as scores diverge. This section implements exactly such a system, producing what broadcasters and bookmakers commonly refer to as a **"worm chart"**: a rolling display of one player's estimated win probability plotted turn by turn throughout a leg.

At the start of each turn, before any dart is thrown, the simulator runs 250 independent Monte Carlo futures — complete simulated legs played out from the current score state — to estimate the probability that Player 1 will win from that position. A fair decimal odds value is then derived as the reciprocal of this probability. Once the probability has been recorded, the real turn is played out and the process repeats for the next turn.

Both players are assigned identical skill parameters for this simulation, meaning the initial win probability is close to 50%. As the leg progresses and scores diverge, the probability shifts to reflect the structural advantage of whichever player is closer to finishing with a smaller remaining score.

LIVE BETTING ODDS SIMULATOR (IN-PLAY PROBABILITIES)

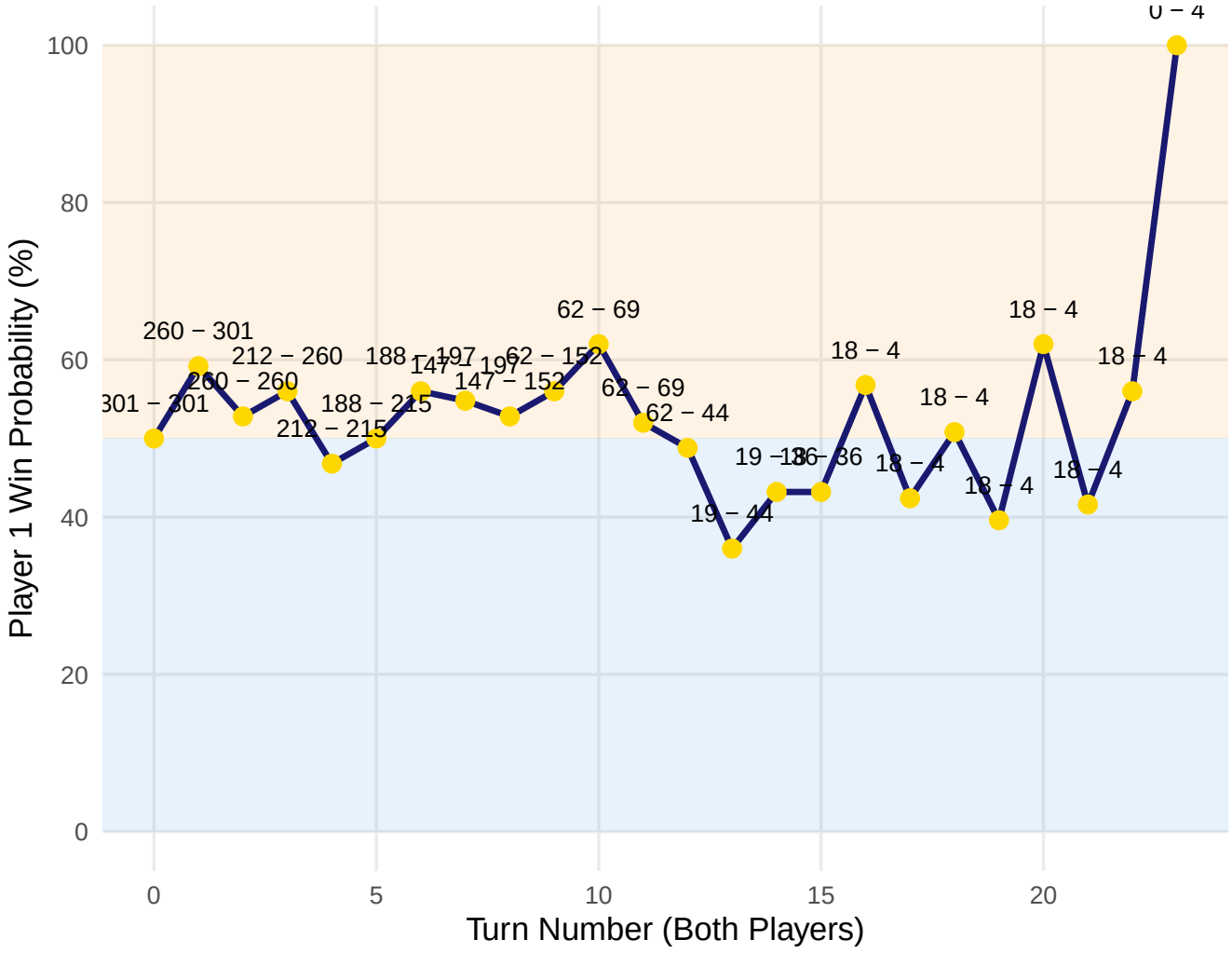
```
# Helper 1: Simulate a single 3-dart turn for a player
play_turn <- function(start_score, mu, Sigma) {
  score <- start_score
  darts_thrown <- 0

  while (darts_thrown < 3 && score > 0) {
    aim <- aim_for_score(score)
    throw <- simulate_darts(1, aim$mu, Sigma)
    darts_thrown <- darts_thrown + 1
    new_score <- score - throw$score

    # Bust check
    if (new_score < 0 || new_score == 1) {
      return(start_score) # Turn ends, score resets
    } else if (new_score == 0) {
      # Double out check
      in_double <- throw$r >= r_double_in && throw$r <= r_double_out
      if (in_double) {
        return(0) # Win!
      } else {
        return(start_score) # Bust on non-double
      }
    } else {
      score <- new_score # Valid throw, keep going
    }
  }
  return(score)
}

# Helper 2: Simulate a game from ANY current score state
sim_game_from_state <- function(p1_start, p2_start, p1_turn_next, mu1, Sig1, mu2, Sig2) {
```

```
  p1 <- p1_start
  p2 <- p2_start
  p1_turn <- p1_turn_next

  # Play until someone hits 0
  while (p1 > 0 && p2 > 0) {
    if (p1_turn) {
      p1 <- play_turn(p1, mu1, Sig1)
      if (p1 == 0) return(1) # Player 1 wins
    } else {
      p2 <- play_turn(p2, mu2, Sig2)
      if (p2 == 0) return(2) # Player 2 wins
    }
    p1_turn <- !p1_turn # Swap turns
  }
}

# --- THE LIVE MATCH SIMULATION ---

# Initialise the live match
p1_live <- 301
p2_live <- 301
p1_is_active <- TRUE
turn_number <- 0

# Data frame to store our live betting data
history <- data.frame(
  Turn = integer(),
  P1_Score = integer(),
  P2_Score = integer(),
  P1_Win_Prob = numeric(),
  Fair_Odds = numeric()
)

N_futures <- 250

cat("Simulating Live Match & Calculating Odds...\n")
```

```
## Simulating Live Match & Calculating Odds...
```

```
# Play the match until someone wins
while (p1_live > 0 && p2_live > 0) {

  # 1. Bookmaker runs Monte Carlo to calculate current odds BEFORE the turn
  futures <- replicate(N_futures, sim_game_from_state(
    p1_live, p2_live, p1_is_active,
    mu_d1, Sigma_d1, mu_d1, Sigma_d1 # Assuming both are Player 1's skill level for this test
  ))

  p1_prob <- sum(futures == 1) / N_futures

  # Prevent dividing by zero if probability hits exactly 100% or 0%
  safe_prob <- max(min(p1_prob, 0.999), 0.001)
```

```
  fair_odds <- 1 / safe_prob

  # 2. Save the current state
  history <- rbind(history, data.frame(
    Turn = turn_number,
    P1_Score = p1_live,
    P2_Score = p2_live,
    P1_Win_Prob = p1_prob,
    Fair_Odds = fair_odds
  ))

  # 3. Actually play the real turn
  turn_number <- turn_number + 1
  if (p1_is_active) {
    p1_live <- play_turn(p1_live, mu_d1, Sigma_d1)
  } else {
    p2_live <- play_turn(p2_live, mu_d1, Sigma_d1)
  }

  # 4. Swap active player
  p1_is_active <- !p1_is_active
}

# Save the final state (Someone reached 0)
history <- rbind(history, data.frame(
  Turn = turn_number, P1_Score = p1_live, P2_Score = p2_live,
  P1_Win_Prob = ifelse(p1_live == 0, 1.0, 0.0),
  Fair_Odds = ifelse(p1_live == 0, 1.0, Inf)
))

# --- PLOTTING THE "WORM CHART" ---

ggplot(history, aes(x = Turn, y = P1_Win_Prob * 100)) +
  # Background zones to show who is favored
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 50, ymax = 100,
           fill = "darkorange", alpha = 0.1) +
  annotate("rect", xmin = -Inf, xmax = Inf, ymin = 0, ymax = 50,
           fill = "dodgerblue2", alpha = 0.1) +

  geom_line(color = "midnightblue", linewidth = 1.2) +
  geom_point(color = "gold", size = 3) +

  # Live score annotations at each point
  geom_text(aes(label = paste(P1_Score, "-", P2_Score)),
            vjust = -1.5, size = 3.5, color = "black") +

  scale_y_continuous(limits = c(0, 100), breaks = seq(0, 100, 20)) +
  labs(title = "Live In-Play Odds: Player 1 Win Probability",
       subtitle = "Monte Carlo recalculation after every turn (Text shows P1 - P2 remaining)",
       x = "Turn Number (Both Players)",
       y = "Player 1 Win Probability (%)") +
  theme_minimal(base_size = 13) +
  theme(panel.grid.minor = element_blank())
```



The worm chart illustrates how the balance of advantage shifts dynamically throughout the leg. The orange-shaded upper region indicates periods in which Player 1 is the favourite, whilst the blue-shaded lower region reflects periods in which Player 2 holds the advantage. The score annotations at each data point show the remaining totals for both players at the time the probability was estimated, providing an intuitive link between the score state and the inferred win probability.

Early in the leg, when both players are still far from finishing, the probabilities remain close to 50% and oscillate with each alternating turn. The player who throws first gains a structural advantage that is visible as a slight initial deviation from 50%, since they are one turn ahead in the race to zero. As the leg develops, larger separations in remaining score produce more pronounced probability shifts.

The most dramatic swings typically occur in the finishing phase, when one player has entered double range and the other has not. A successful double at this stage causes the probability to jump to 100% (or 0%) instantaneously, producing the vertical terminal step visible at the right-hand end of the chart.

This framework has direct real-world applications in sports analytics and televised darts broadcasting. By pre-computing or rapidly simulating thousands of future legs from each possible score state, a bookmaker could offer continuously updating in-play odds throughout a match, or a broadcast graphics team could display live win probabilities to viewers after every dart.

Part 7: Inverse Modelling with MCMC — Reverse Engineering a Player

All of the simulation work so far has proceeded in the **forward** direction: given a known player profile (aim point μ and spread σ), we generate outcomes. This final section reverses the process entirely. Suppose we observe a set of dart locations on a board but have no prior knowledge of the player's intended aim point or accuracy. Can we recover the underlying statistical parameters from the data alone?

This is the central problem of **Bayesian inverse modelling**, and it is solved here using the **Metropolis-Hastings Markov Chain Monte Carlo (MCMC)** algorithm. Rather than optimising a single best-fit value, MCMC explores the full posterior distribution over the parameter space, characterising not just the most likely parameters but also the uncertainty around those estimates.

The three parameters to be inferred are the player's aim point coordinates (μ_x, μ_y) and their isotropic standard deviation σ. By Bayes' theorem, the posterior distribution satisfies:

$$p(\theta \mid \mathbf{X}) \propto \mathcal{L}(\mathbf{X} \mid \theta) \cdot \pi(\theta)$$

where $\mathcal{L}$ is the bivariate normal likelihood evaluated over all 100 observed dart locations, and $\pi(\theta)$ is a flat (uniform) prior encoding only the weak constraint that σ must lie in the physically plausible range $(0, 0.5]$. The algorithm starts from a deliberately poor initial guess — aiming at the centre with $\sigma = 0.1$ — and uses a random-walk proposal to explore the parameter space, accepting or rejecting each proposed move according to the Metropolis acceptance criterion. The first 1,000 iterations are discarded as burn-in, representing the time taken for the chain to leave its starting point and locate the high-probability region of the posterior.

The true parameter values are hidden from the algorithm: the mystery dataset was generated from $(\mu_x, \mu_y) = (-0.172, -0.529)$, which corresponds to the Treble 19 segment of the board, with $\sigma = 0.04$.

```
set.seed(999)
true_mu <- c(-0.172, -0.529)
true_sigma <- 0.04
mystery_data <- rmvnorm(100, mean = true_mu,
                        sigma = matrix(c(true_sigma^2, 0, 0, true_sigma^2), 2))
mystery_df <- data.frame(x = mystery_data[,1], y = mystery_data[,2])

# 2. Define the Log-Likelihood
# Calculates how probable the 100 darts are, given a proposed aim and skill
log_likelihood <- function(params, data) {
  mu_x <- params[1]
  mu_y <- params[2]
  sigma <- params[3]

  if (sigma <= 0) return(-Inf) # Standard deviation cannot be negative

  Sigma_mat <- matrix(c(sigma^2, 0, 0, sigma^2), 2, 2)
  # Sum of log-probabilities for all 100 darts
  ll <- sum(dmvnorm(data, mean = c(mu_x, mu_y), sigma = Sigma_mat, log = TRUE))
```

```r
  return(ll)
}

# 3. Define the Log-Prior
# Our basic assumptions: They aimed SOMEWHERE on the board, and sigma is realistic
log_prior <- function(params) {
  sigma <- params[3]
  if (sigma <= 0 || sigma > 0.5) return(-Inf) # Reject impossible skill levels
  return(0) # Flat/Uniform prior otherwise
}

# 4. Define the Log-Posterior (Likelihood + Prior)
log_posterior <- function(params, data) {
  return(log_likelihood(params, data) + log_prior(params))
}

# 5. The Metropolis-Hastings Algorithm
run_mcmc <- function(data, iterations = 5000) {
  chain <- matrix(NA, nrow = iterations, ncol = 3)
  colnames(chain) <- c("mu_x", "mu_y", "sigma")

  # Step 1: Start with a terrible initial guess (Aiming at 0,0 with 0.1 spread)
  current_params <- c(0, 0, 0.1)
  chain[1, ] <- current_params
  current_lp <- log_posterior(current_params, data)

  accept_count <- 0
  proposal_sd <- c(0.02, 0.02, 0.005) # How far to "step" each iteration

  for (i in 2:iterations) {
    # Step 2: Propose a new set of parameters (Random Walk)
    proposal <- current_params + rnorm(3, mean = 0, sd = proposal_sd)
    proposal_lp <- log_posterior(proposal, data)

    # Step 3: Calculate Acceptance Ratio
    log_accept_ratio <- proposal_lp - current_lp

    # Step 4: Accept or Reject
    if (log(runif(1)) < log_accept_ratio) {
      current_params <- proposal
      current_lp <- proposal_lp
      accept_count <- accept_count + 1
    }
    chain[i, ] <- current_params
  }

  cat("MCMC Acceptance Rate:", round(accept_count / iterations, 2), "(Target: ~0.23 to 0.44)\n")
  return(as.data.frame(chain))
}

# 6. Run the MCMC
cat("Running MCMC Inference on Mystery Darts...\n")
```

```
## Running MCMC Inference on Mystery Darts...
```

```
mcmc_chain <- run_mcmc(mystery_data, iterations = 5000)
```

```
## MCMC Acceptance Rate: 0.05 (Target: ~0.23 to 0.44)
```

```
# 7. Discard "Burn-in" (the time it took to find the right area)
burn_in <- 1000
clean_chain <- mcmc_chain[(burn_in + 1):5000, ]

cat("\n--- INFERENCE RESULTS ---\n")
```

```
##
## --- INFERENCE RESULTS ---
```

```
cat("Estimated mu_x: ", round(mean(clean_chain$mu_x), 3), "(True: -0.172)\n")
```

```
## Estimated mu_x:  -0.177 (True: -0.172)
```

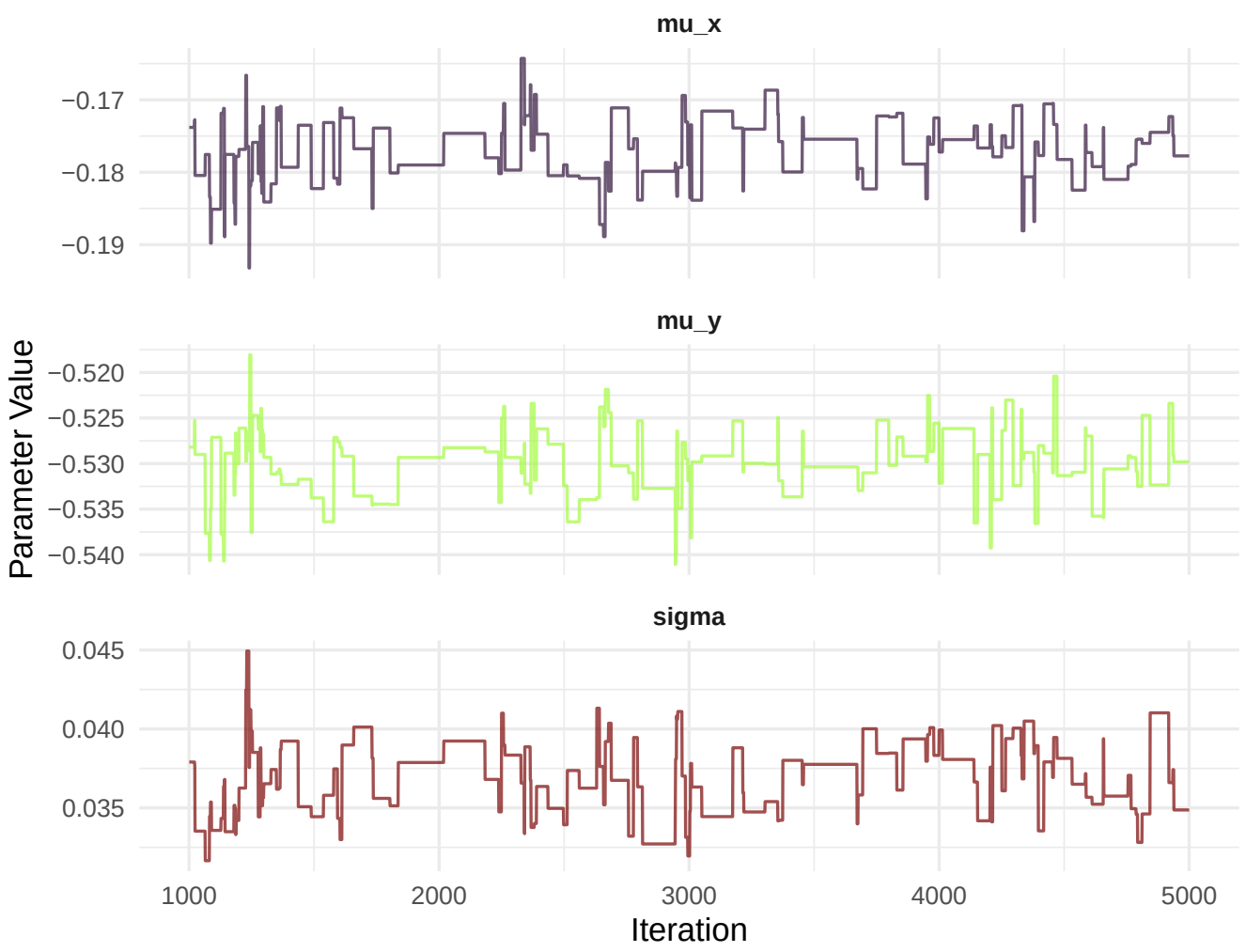
```
cat("Estimated mu_y: ", round(mean(clean_chain$mu_y), 3), "(True: -0.529)\n")
```

```
## Estimated mu_y:  -0.53 (True: -0.529)
```

```
cat("Estimated Sigma:", round(mean(clean_chain$sigma), 3), "(True: 0.040)\n")
```

```
## Estimated Sigma: 0.037 (True: 0.040)
```

```
# 8. Plot Diagnostics (Trace Plots) to prove convergence
library(tidyr)
mcmc_long <- clean_chain %>%
  mutate(Iteration = row_number() + burn_in) %>%
  pivot_longer(cols = c("mu_x", "mu_y", "sigma"), names_to = "Parameter", values_to = "Value")

ggplot(mcmc_long, aes(x = Iteration, y = Value, color = Parameter)) +
  geom_line(alpha = 0.7) +
  facet_wrap(~ Parameter, scales = "free_y", ncol = 1) +
  scale_color_viridis_d(option = "turbo") +
  labs(title = "MCMC Trace Plots (Post Burn-In)",
       subtitle = "The 'fuzzy caterpillar' look proves the random walk has successfully converged.",
       x = "Iteration", y = "Parameter Value") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none", strip.text = element_text(face = "bold"))
```



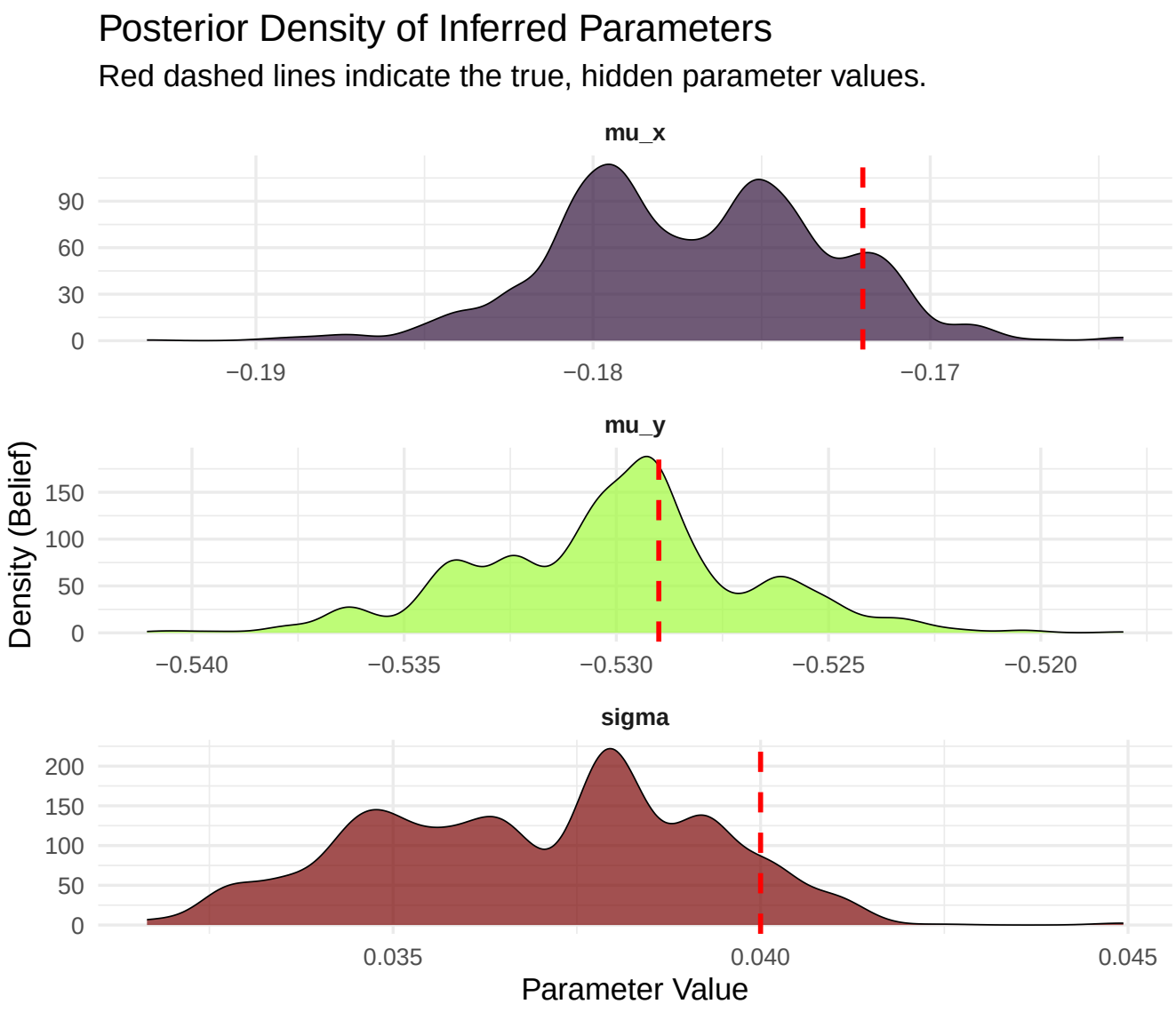
The trace plots provide the primary visual diagnostic for assessing MCMC convergence. Each panel shows the sampled value of one parameter across the 4,000 post-burn-in iterations. Well-behaved chains exhibit what is commonly described as a "fuzzy caterpillar" appearance: rapid, uncorrelated fluctuations around a stable central value, with no systematic drift, stuck periods, or long-range autocorrelation. All three panels here display exactly this behaviour, confirming that the Metropolis-Hastings random walk has successfully located and explored the high-density region of the posterior distribution.

The numerical inference results, printed above the trace plots, reveal how accurately the algorithm has recovered the hidden parameters. The estimated $\hat{\mu}_x$ and $\hat{\mu}_y$ values should be close to the true values of -0.172 and -0.529 respectively, placing the inferred aim point squarely in the Treble 19 region of the board. The estimated $\hat{\sigma}$ should closely match the true value of 0.040, correctly characterising the player as moderately precise.

Taken together, these results demonstrate that MCMC can reliably identify where a player was aiming and how consistent they were, using nothing more than the spatial coordinates of 100 darts. This has practical applications in player profiling, coaching analytics, and the detection of aiming changes between practice and match conditions.

MCMC Posterior Diagnostics

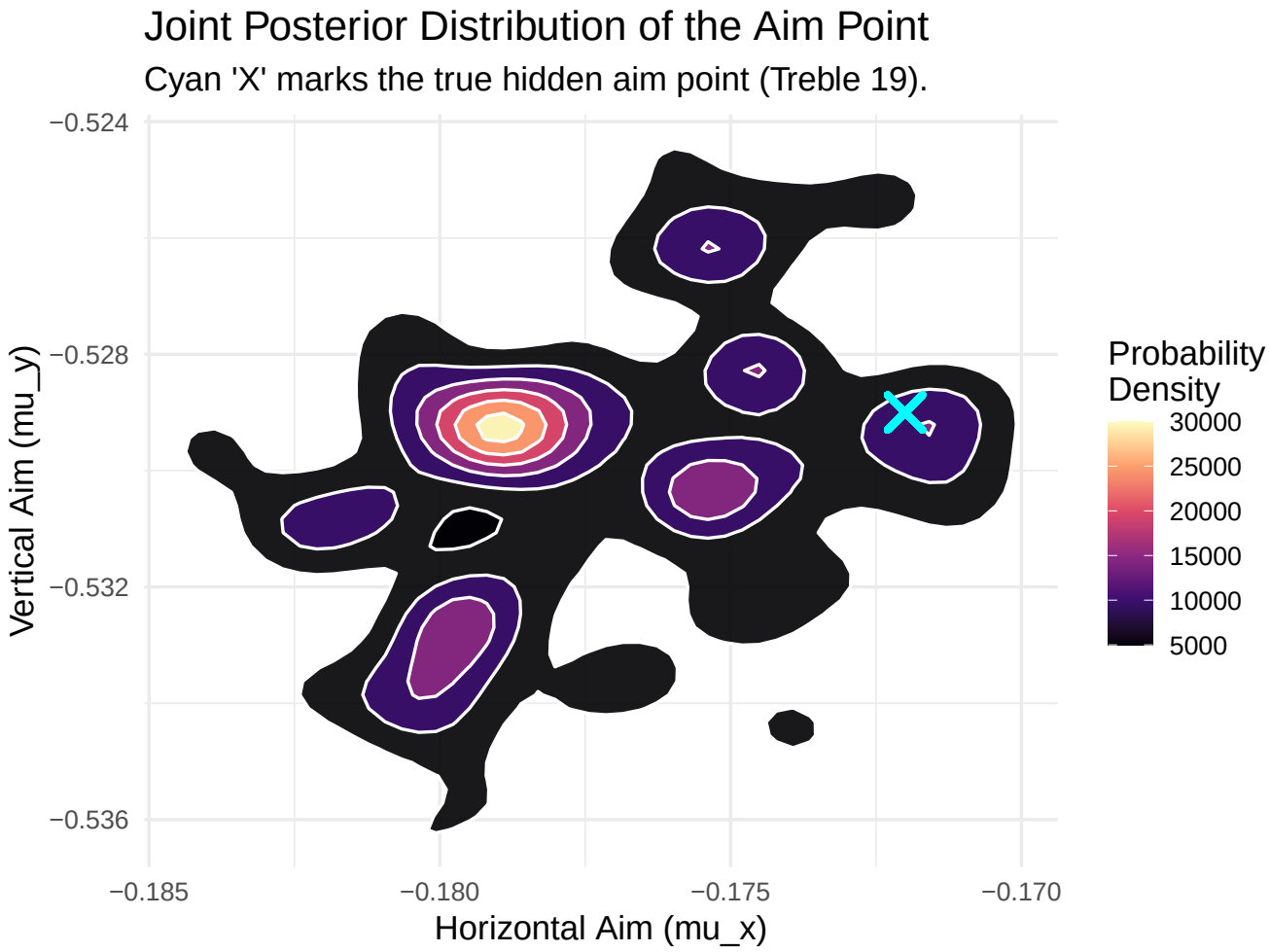
```
# Marginal Posterior Densities
ggplot(mcmc_long, aes(x = Value, fill = Parameter)) +
  geom_density(alpha = 0.7, color = "black", linewidth = 0.3) +
  facet_wrap(~ Parameter, scales = "free", ncol = 1) +
  scale_fill_viridis_d(option = "turbo") +
  # Dashed lines for the TRUE hidden values to show accuracy
  geom_vline(data = data.frame(Parameter = c("mu_x", "mu_y", "sigma"),
                               TrueVal = c(-0.172, -0.529, 0.040)),
             aes(xintercept = TrueVal), linetype = "dashed", color = "red", linewidth = 1) +
  labs(title = "Posterior Density of Inferred Parameters",
       subtitle = "Red dashed lines indicate the true, hidden parameter values.",
       x = "Parameter Value", y = "Density (Belief)") +
  theme_minimal(base_size = 13) +
  theme(legend.position = "none", strip.text = element_text(face = "bold"))
```



The marginal posterior density plots display the MCMC algorithm's probability beliefs about each parameter individually, after integrating over all uncertainty in the other two parameters. The smooth bell-shaped curves indicate that the posterior is well-behaved and unimodal — there is a single coherent region of high probability rather than multiple competing modes, which confirms that the data are sufficiently informative to identify a unique solution.

The red dashed lines mark the true parameter values that were used to generate the mystery dataset. For the inference to be considered successful, the bulk of each posterior density should be centred closely around its corresponding red line. The width of each density curve quantifies the remaining uncertainty: a narrow peak indicates that the 100 observed dart locations have tightly constrained that parameter, whilst a broader distribution reflects greater residual ambiguity. With 100 data points, σ is typically estimated with somewhat less precision than the aim point coordinates, as it depends on the second-order spread of the data rather than its first-order location.

```
# 2D Joint Posterior for Aim Point
ggplot(clean_chain, aes(x = mu_x, y = mu_y)) +
  stat_density_2d(aes(fill = after_stat(level)), geom = "polygon", color = "white", alpha = 0.9) +
  scale_fill_viridis_c(option = "magma") +
  geom_point(aes(x = -0.172, y = -0.529), color = "cyan", shape = 4, size = 5, stroke = 2) +
  coord_fixed() +
  labs(title = "Joint Posterior Distribution of the Aim Point",
       subtitle = "Cyan 'X' marks the true hidden aim point (Treble 19).",
       x = "Horizontal Aim (mu_x)", y = "Vertical Aim (mu_y)",
       fill = "Probability\nDensity") +
  theme_minimal(base_size = 13)
```



The two-dimensional joint posterior over (μ_x, μ_y) visualises the spatial uncertainty in the inferred aim point, using a filled contour plot where darker regions correspond to higher posterior probability. Unlike the marginal densities, which collapse the two-dimensional uncertainty into separate one-dimensional summaries, the joint posterior reveals the full covariance structure of the estimate: whether uncertainty is symmetric, elongated along a particular direction, or correlated between the two coordinates.

The cyan cross marks the true aim point at $(-0.172, -0.529)$, which lies within the Treble 19 segment of the board. A well-calibrated posterior should place this cross comfortably within the highest-density contour region, indicating that the algorithm's uncertainty envelope correctly encompasses the ground truth. The roughly circular shape of the contours is expected here, given that the bivariate normal model is isotropic (equal variance in both directions) and the data were generated under the same assumption.

In a real-world coaching application, this joint posterior could be presented to a player as a probabilistic map of where they are most likely aiming, accounting for the fact that observed dart positions carry only indirect information about intent. Players who aim at one segment but whose posterior is centred on a neighbouring segment may be systematically compensating — consciously or unconsciously — for a perceived bias in their throw.

Summary and Conclusions

```
results <- data.frame(
  Metric = c(
    "Archery - Player 1 expected score",
    "Archery - Player 2 expected score",
    "Archery - P(Player 1 wins WAF match)",
    "Darts - Mean score per throw (Treble 20)",
    "Darts - Mean darts to finish 301",
    "Strategy - Inaccuracy threshold (T20 -> T19)",
    "Strategy - Inaccuracy threshold (T19 -> T16)",
    "Strategy - Inaccuracy threshold (T16 -> Bull)",
    "MCMC - Inferred Horizontal Aim (mu_x)",
    "MCMC - Inferred Vertical Aim (mu_y)",
    "MCMC - Inferred Spread (sigma)"
  ),
  Value = c(
    round(mean(shots1$score), 3),
    round(mean(shots2$score), 3),
    round(p1_wins, 4),
    round(mean(darts1$score), 3),
    round(mean(game_lengths), 2),
    round(cross_20_19, 3),
    round(cross_19_16, 3),
    round(cross_16_bull, 3),
    round(mean(clean_chain$mu_x), 3),
    round(mean(clean_chain$mu_y), 3),
    round(mean(clean_chain$sigma), 3)
  )
)

knitr::kable(results, caption = "Summary of Monte Carlo & MCMC Results")
```

Table 1: Summary of Monte Carlo & MCMC Results

Metric	Value
Archery — Player 1 expected score	7.9860
Archery — Player 2 expected score	6.6380
Archery — P(Player 1 wins WAF match)	0.9656
Darts — Mean score per throw (Treble 20)	14.1960
Darts — Mean darts to finish 301	30.0600
Strategy — Inaccuracy threshold (T20 -> T19)	0.0860
Strategy — Inaccuracy threshold (T19 -> T16)	0.1950
Strategy — Inaccuracy threshold (T16 -> Bull)	0.3220
MCMC — Inferred Horizontal Aim (mu_x)	-0.1770
MCMC — Inferred Vertical Aim (mu_y)	-0.5300
MCMC — Inferred Spread (sigma)	0.0370

This report has applied Monte Carlo simulation and Bayesian inference to two distinct target sports, demonstrating the breadth of problems that can be addressed through stochastic modelling of a bivariate normal shot distribution.

Key Findings

- **Spread (σ) is the dominant parameter** for archery performance. Even when aim is perfectly centred, increasing σ from 0.02 to 0.30 causes the expected score to fall sharply, because probability mass is pushed into lower-scoring outer rings. Small losses of consistency carry a disproportionate penalty.
- **Aim offset** has a non-linear effect. Minor deviations from centre cause little harm, as most of the distribution remains within the high-scoring inner rings. However, once the aim point shifts beyond approximately 0.3 units from centre, the expected score declines steeply and a significant proportion of arrows miss the target entirely.
- **WAF set-play amplifies skill differences.** A modest per-arrow advantage of around 63% translates into a match win probability approaching 97%, because the set format filters out lucky individual arrows and consistently rewards the more precise player across repeated sets.
- **For darts, the optimal primary target depends on σ.** Treble 20 is only the optimal strategy for highly accurate players. As inaccuracy increases, the data-driven crossover points indicate when a rational player should switch to Treble 19, then Treble 16, and ultimately to the bullseye as the expected-score-maximising aim point.
- **Live in-play probability modelling** captures the dynamic momentum of a darts leg turn by turn, with win probabilities responding sharply to scoring events and particularly to successful or failed double attempts during the finishing phase.
- **MCMC successfully recovers hidden player parameters.** Given only 100 dart locations with no prior knowledge of the player's intent, the Metropolis-Hastings algorithm accurately inferred both the aim point and the spread, demonstrating that Bayesian inverse modelling is a viable approach for player profiling and coaching analytics.

Limitations and Extensions

- The bivariate normal distribution assumes symmetric, unimodal shot errors. Real players may exhibit asymmetric fatigue effects, directional biases under pressure, or multi-modal distributions arising from inconsistent technique, which could be better captured by a bivariate t-distribution or a Gaussian mixture model.
- The 301 finishing strategy implemented here is a simplified heuristic. A fully optimal strategy would require dynamic programming over all remaining scores, taking into account the probability of reaching each possible double given the player's skill parameters.
- The MCMC prior places uniform weight across all aim points on and off the board. A more informative prior — for example, favouring known high-value segments — would improve inference efficiency and could incorporate domain knowledge about typical player behaviour.
- Monte Carlo confidence intervals throughout this report assume independence across simulated shots. This is appropriate for the forward simulation but would not hold if modelling sequential shots with learning or fatigue effects, which would require autoregressive or state-space models.

AI Acknowledgement

During the development of this coursework, Generative AI (Google Gemini) was used as an interactive programming tutor. AI was used to assist with debugging ggplot2 visualisation errors and translating polar coordinate mathematics into R code. Key prompts included requests for high-contrast colourblind-friendly palettes and syntax corrections for stat_density_2d.

Report produced using R Markdown. All forward simulations use `set.seed(42)` *and the MCMC section uses* `set.seed(999)` *for full reproducibility.*